
ByteFloats.jl

0.1.0

ByteFloats.jl contributors

July 18, 2026

Contents

Contents	ii
1 ByteFloats.jl	1
1.1 Documentation map	1
2 Introduction	2
2.1 Why this package exists	2
2.2 Thirty-second tour	3
2.3 The formats	3
2.4 Where to go next	3
2.5 Installation	3
2.6 Status and conformance	4
3 User Guide	5
3.1 Formats and format queries	5
3.2 Values	6
3.3 Projection specifications	7
3.4 Scalar operations: the two registers	8
3.5 Arrays, kernels, and sorting	9
3.6 Blocks and scaled operations	9
3.7 Packed storage	10
3.8 Conformance and κ -approximate implementations	10
3.9 Performance guidance	10
3.10 Environment switches	11
4 User Examples	12
4.1 Basic	12
4.2 General AI	14
4.3 Machine Learning	16
4.4 Deep Learning	17
5 Technical Guide	21
5.1 Layer map	21
5.2 Encoding and decoding	21
5.3 The projection engine	22
5.4 The oracle and the result-kind protocol	22
5.5 Tables and kernels	23
5.6 Blocks: exactness without a superaccumulator	23
5.7 The κ registry and conformance	23
5.8 Verification doctrine	24
5.9 Benchmark doctrine	24
5.10 Deliberate limitations	24
6 Technical Examples	25
6.1 Basic	25
6.2 General AI	27
6.3 Machine Learning	29
6.4 Deep Learning	31

7	Adding Operations	34
7.1	Universal duties (every new operation)	34
7.2	Scalar operations	35
7.3	Block operations	39
7.4	After any addition: the closing checklist	41
8	API Reference	42
8.1	Public API	42
8.2	Documented internals	48

Chapter 1

ByteFloats.jl

A conforming, performance-oriented Julia implementation of the IEEE P3109 draft standard — arithmetic formats for machine learning at bitwidths 3–8.

Bit-exact defined results on every default path; one projection engine as the single write path into a code point; approximation only behind an explicitly named, exhaustively measured κ registry; ≈ 8.8 million test assertions; table-gather kernels at fractions of a nanosecond per element.

```
julia> using ByteFloats

julia> Binary8p4se(1.6) + Binary8p4se(0.25)
Binary8p4se(1.875 == 0x47)

julia> Binary5p3sf(0x08) == Binary5p3sf(1.0)    # UInt8 constructs from a code point
true
```

1.1 Documentation map

- **Introduction** — what the package is, the design pillars, a thirty-second tour, installation.
- **User Guide** — the complete public API in usage order: formats, values, projection specifications, the two operation registers, arrays and sorting, blocks, packed storage, conformance and κ , performance guidance.
- **User Examples** — runnable examples in three tiers: basic, machine learning, deep learning.
- **Technical Guide** — internals: the encoding and projection engine, the oracle's rigor classes, tables and kernels, the block layer's exactness filters, the verification and benchmark doctrines.
- **Technical Examples** — internals-level recipes: pipeline introspection, exhaustive verification of custom code, κ measurement, doctrine-compliant benchmarking.
- **Adding Operations** — how to add new scalar (unary through quaternary) and block (scaled, elementwise, reductive) operations with every guarantee intact: registry mechanics, ω -semantics duties, worked examples.
- **API Reference** — docstrings for the documented public surface.

Chapter 2

Introduction

ByteFloats.jl is a conforming, performance-oriented Julia implementation of the **IEEE P3109 draft standard** — *Arithmetic Formats for Machine Learning* — covering every binary format the draft defines at bitwidths 3 through 8, every projection (rounding × saturation) mode including the three stochastic families, the full scalar operation catalog, block-scaled ("MX-style") operations, and the draft's conformance and κ -approximation machinery.

2.1 Why this package exists

Small floating-point formats are easy to implement *approximately* and surprisingly hard to implement *exactly*. An FP8 Exp differs from a bit-exact one only in a handful of code points near rounding boundaries — invisible in a demo, decisive in a conformance suite, and quietly corrosive in research that compares number formats. ByteFloats.jl takes the position that for value sets this small there is no excuse for approximation:

- **Bit-exact defined results on every default path.** Every operation's result is the correct projection of the mathematically exact value, established through a rigorous oracle (exact arithmetic, correctly-rounded Float128 where IEEE mandates it, and MPFR directed-rounding enclosures with automatic precision escalation everywhere else).
- **One projection engine.** RoundToPrecision → Saturate → Encode is the *single* write path into a code point. There is no second, "fast but slightly different" rounding routine anywhere in the package.
- **Approximation is opt-in, named, and measured.** Faster-but-inexact implementations live behind an explicit registry ([register_approx!](#)); each registration's deviation bound κ is *measured by exhaustive enumeration* at registration time, and understated declarations are rejected. Nothing approximate is reachable from the default API.
- **Exhaustively tested.** The value sets are tiny (≤ 256 points per format), so the test suite does not sample — it enumerates: every operation on every input, every ordering of every pair, every stochastic draw at boundary budgets. The shipped suite carries ≈ 8.8 million assertions.
- **Fast where it matters.** Pure-mode elementwise work runs through precomputed lookup tables (sub-nanosecond per element); the scalar path is fully specialized and allocation-free (≈ 18 ns for a complete Add including projection); sub-byte packed storage, integer-keyed ordering with an $O(n)$ counting sort, and a mask-based rounding core round out the performance story. A reproducible Chairmarks benchmark suite ships in `benchmark/`.

2.2 Thirty-second tour

```
julia> using ByteFloats

julia> x = Binary8p4se(1.6)           # construct = project under the default spec
Binary8p4se(1.625 == 0x45)

julia> x + Binary8p4se(0.25)         # Base operators use the format's default spec
Binary8p4se(1.875 == 0x47)

julia> Add(Binary8p4se, ProjSpec(TowardPositive(), SatNone()), x, Binary8p4se(0.25))
Binary8p4se(1.875 == 0x47)           # spec-named register: any mode, explicitly

julia> σ = ProjSpec(StochasticA{8}(), SatNone()); # stochastic rounding, N = 8 bits

julia> Add(Binary8p4se, σ, Binary8p4se(2.0), Binary8p4se(0.03125)); R = 255)
Binary8p4se(2.25 == 0x49)           # explicit draw R makes it reproducible

julia> Exp(Binary8p4se, RNE_SatNone, Binary8p4se.([-1.0, 0.5, 2.0]))
3-element Vector{Binary8p4se}:      # array methods route through 256-byte tables
  Binary8p4se(0.375 == 0x34)
  Binary8p4se(1.625 == 0x45)
  Binary8p4se(7.5 == 0x57)
```

2.3 The formats

A format is the type `Binary{K,P,SGN,EXT}`: bitwidth $K \in 3:8$, precision $P \in 1:K$ (with the signedness constraint), signed/unsigned `SGN`, and extended (has $\pm\text{Inf}$) or finite `EXT`. All 120 draft formats carry their draft names as exported aliases — `Binary8p4se` is `Binary{8,4,true,true}` (" $K = 8$, $P = 4$, signed, extended"); the trailing letters are `s/u` for signedness and `e/f` for extended/finite. Values are 1-byte immutable wrappers around their code point; every format has a single NaN, no negative zero, and `Float64` serves as the exact interchange carrier for all datums.

2.4 Where to go next

- **User Guide** — the complete tour of the public API: formats and values, projection specifications, the two operation registers, arrays and sorting, blocks and scaled operations, packed storage, conversion, conformance, and performance guidance.
- **User Examples** — worked, runnable examples in three tiers: basic usage, machine-learning quantization workflows, and deep-learning block/kernel patterns.
- **Technical Guide** — the internals: the encoding and projection engine, the oracle's evaluation protocol and its two rigor classes, tables and kernels, the block layer's exactness filters, and the testing and benchmarking doctrine.
- **Technical Examples** — internals-level recipes: pipeline introspection, verifying custom kernels against the oracle, κ measurement, differential builds, and doctrine-compliant benchmarking.

2.5 Installation

```
pkg> dev path/to/ByteFloats.jl # from source (the package is not yet registered)
```

Requires Julia 1.12. Quadmath (libquadmath's Float128) is a hard dependency used only inside the oracle and exact-fallback paths; on platforms where it misbehaves, `ENV["ByteFloats_Float128"] = "disable"` before loading selects the pure-MPFR configuration with **identical results** (this equivalence is itself part of the test suite).

2.6 Status and conformance

`conformance()` returns the live, serializable conformance declaration (formats, operations, modes, instantiated table specializations, and `κ` registrations); `ByteFloats.draft_revision()` names the implemented draft revision. Interpretations made where the draft text under-determined behavior are marked `[interp]` in the source comments.

Chapter 3

User Guide

Everything you need to use ByteFloats.jl, in the order you'll need it. All transcripts in this guide are captured from real sessions against the shipped package.

3.1 Formats and format queries

The parametric type `Binary{K,P,SGN,EXT}` defines a format: bitwidth $K \in 3:8$, precision P (significand bits including the implicit bit), `SGN::Bool` for signed/unsigned, `EXT::Bool` for extended (with $\pm\text{Inf}$) versus finite. Every legal combination — 120 formats — has its draft name exported:

```
julia> Binary8p4se === Binary{8,4,true,true}
true

julia> formatname(Binary{6,3,true,false})
:Binary6p3sf
```

The suffix reads: p4 precision 4, s/u signed/unsigned, e/f extended/finite.

Format introspection follows the draft's Group M, with both Julia-style and draft-named accessors:

```
julia> bitwidth(Binary8p4se), precision(Binary8p4se), expbias(Binary8p4se)
(8, 4, 8)

julia> MaxFiniteOf(Binary8p4se)
Binary8p4se(224.0 ≡ 0x7e)

julia> MinPositiveOf(Binary8p4se)
Binary8p4se(0.0009765625 ≡ 0x01)
```

Also available: `MinFiniteOf`, `MinNormalOf`, `MaxSubnormalOf`, `expbitwidth`, `trailingsigbits`, `issigned`, `isextended`, and the draft-named forms (`BitwidthOf`, `PrecisionOf`, `ExponentBiasOf`, ...).

3.2 Values

A value is one byte: an immutable wrapper around its code point. Four ways in, two ways out:

```
julia> x = Binary8p4se(1.6)           # construct from a Real: projects (rounds)
Binary8p4se(1.625 ≡ 0x45)

julia> Binary8p4se(0x45)             # construct from a UInt8 CODE POINT (validated)
Binary8p4se(1.625 ≡ 0x45)

julia> rawvalue(Binary8p4se, 0x45)   # same, unchecked – the kernel-internal route
Binary8p4se(1.625 ≡ 0x45)

julia> Convert{Binary8p4se, RNE_SatNone, 3}(1.6) # explicit conversion, any mode
Binary8p4se(3.0 ≡ 0x4c)

julia> decode(x)                     # exact datum as Float64 (always exact)
1.625

julia> codepoint(x)                  # the raw byte (extends Base.codepoint)
0x45
```

UInt8 means code point; every other number means value

UInt8 is the *only* argument type with code-point semantics — mirroring Char(0x41). Every other Real (including other Integers) constructs by projecting the numeric value, and Convert is numeric for **all** integers:

```
julia> Binary8p4se(0x02), Binary8p4se(2)
(Binary8p4se(0.001953125 ≡ 0x02), Binary8p4se(2.0 ≡ 0x48))
```

Out-of-range codes throw for $K < 8$ (Binary5p3sf(0x20) is an error); the range check costs nothing measurable — 2.1 ns, identical to unchecked rawvalue. Round-tripping is $T(\text{codepoint}(x)) == x$.

Convert accepts Binary values (any format), Float16/32/64, Integer, and BigFloat — types whose values it can project *exactly*. For other Reals, convert explicitly first (e.g. Binary8p4se(Float64(π))) and own the double rounding; Rational inputs throw with that guidance rather than double-round silently.

Every format has exactly one NaN and no negative zero; extended formats add $\pm\text{Inf}$:

```
julia> Binary8p4se(1e9)               # overflow under the default spec → +Inf
Binary8p4se(Inf ≡ 0x7f)

julia> Binary8p4se(-0.0)              # no -0: projects to the single zero
Binary8p4se(0.0 ≡ 0x00)

julia> isnan(Binary8p4se(NaN)), isfinite(Binary8p4se(2.0))
(true, true)
```

Standard predicates (isnan, isinf, isfinite, iszero, signbit, issubnormal), zero/one/eps/typemin/typemax, and stepping via NextGreaterThan / NextLessThan (the draft's Next operations; nextfloat/prevfloat alias them) all work:

```
julia> NextGreaterThan(Binary8p4se(1.0))
Binary8p4se(1.125 == 0x41)

julia> Class(Binary8p4se(0.01))           # draft classification
ClassPosNormal::FPClass = 0x06
```

3.3 Projection specifications

Every rounding decision in the package is governed by a ProjSpec, the pair of a **rounding mode** and a **saturation mode**. Both are zero-size singleton types, so a ProjSpec costs nothing and fully specializes every call it reaches.

Rounding modes: NearestTiesToEven, NearestTiesToAway, TowardPositive, TowardNegative, TowardZero, ToOdd, and the stochastic families StochasticA{N}, StochasticB{N}, StochasticC{N} with a random-bit budget $1 \leq N \leq 60$. Saturation modes: SatFinite, SatPropagate, SatNone.

```
julia> p = ProjSpec(TowardPositive(), SatFinite())
(TowardPositive, SatFinite)

julia> RNE_SatNone, RNE_SatFinite           # the two common specs, predefined
((NearestTiesToEven, SatNone), (NearestTiesToEven, SatFinite))
```

Rounding chooses the neighbor; saturation decides what out-of-range results become. Watch all three interact on an overflowing product in Binary8p4se (MaxFinite = 224):

```
julia> w, two = Binary8p4se(200.0), Binary8p4se(2.0);

julia> Multiply(Binary8p4se, RNE_SatNone, w, two)           # overflow → ±Inf
Binary8p4se(Inf == 0x7f)

julia> Multiply(Binary8p4se, RNE_SatFinite, w, two)         # overflow → MaxFinite
Binary8p4se(224.0 == 0x7e)

julia> Multiply(Binary8p4se, ProjSpec(TowardZero(), SatNone()), w, two)
Binary8p4se(224.0 == 0x7e) # SatNone + directed-toward-zero clamps finite
```

Note

SatNone is not "no saturation handling": it is the draft's row set in which nearest modes overflow to $\pm\text{Inf}$ (or NaN for finite formats) while directed modes that point away from the overflow clamp to the extremal finite value.

Stochastic rounding

Stochastic modes consume N random bits per projection. You control the source three ways — implicitly (task-local RNG), by passing an rng, or by fixing the draw R itself, which makes single projections exactly reproducible and is the right tool for tests:

```
julia> σ = ProjSpec(StochasticA{8}(), SatNone());

julia> Add(Binary8p4se, σ, Binary8p4se(2.0), Binary8p4se(0.03125)); rng = Xoshiro(1))
Binary8p4se(2.0 ≡ 0x48)

julia> Add(Binary8p4se, σ, Binary8p4se(2.0), Binary8p4se(0.03125); R = 0)
Binary8p4se(2.0 ≡ 0x48)      # smallest draw: rounds down

julia> Add(Binary8p4se, σ, Binary8p4se(2.0), Binary8p4se(0.03125); R = 255)
Binary8p4se(2.25 ≡ 0x49)      # largest draw: rounds up
```

The exact fraction here is $1/8$ of an ulp, so over all 256 draws exactly 32 round up — `StochasticA` is unbiased in expectation. R must lie in $0:2^N-1$.

3.4 Scalar operations: the two registers

The spec-named register exposes every draft operation under its draft name with an explicit result format and spec: `Op(fr, ρ, operands...)`. Thirty unary operations (`Abs`, `Negate`, `Sqrt`, `RSqrt`, `Recip`, `Exp`, `Exp2`, `ExpMinusOne`, `Log`, `Log2`, `LogOnePlus`, `Softplus`, the trig/hyperbolic families, the π -scaled families), eighteen binary (`Add`, `Subtract`, `Multiply`, `Divide`, `CopySign`, `Hypot`, `ArcTan2`, `ArcTan2Pi`, and the eight `Minimum/Maximum` variants plus the `Finite` variants), three ternary (`FMA`, `FAA`, `Clamp`), and `Convert`. Operands may be any formats; the result format is the first argument.

The Base register makes each format an ordinary Julia number under its *default* spec (`RNE_SatNone`): `+` `-` `*` `/`, `fma`, `exp`, `log`, `sqrt`, `min`, `max`, `abs`, `atan(y, x)`, `sinpi`, `inv`, comparisons, and friends — same-format operands only, no silent cross-format promotion (mixing formats promotes to `Float64` explicitly).

```
julia> exp(Binary8p4se(0.25))                                # Base register
Binary8p4se(1.25 ≡ 0x42)

julia> Exp(Binary8p4se, RNE_SatNone, Binary8p4se(0.25))    # identical, explicit
Binary8p4se(1.25 ≡ 0x42)
```

Semantics highlights (all per the draft): $x/0 \rightarrow \text{NaN}$ for every x including $\pm\text{Inf}$; $\text{Recip}(\pm\text{Inf}) = 0$; $0 \cdot \infty \rightarrow \text{NaN}$; a NaN operand generally propagates, except the `*Number/*Finite` extremum variants which prefer non-NaN (and finite) operands; trig of $\pm\text{Inf}$ is NaN; the π -scaled family reduces exactly mod 2 first.

3.5 Arrays, kernels, and sorting

Every operation has array methods — `Op(fr, p, A)`, `Op(fr, p, A, B)`, `Op(fr, p, A, B, C)` — plus the generic `vmap / vmap!`:

```
julia> A = Binary8p4se.(randn(Xoshiro(7), 4) .* 2)
4-element Vector{Binary8p4se}:
 Binary8p4se(-0.875 ≐ 0xbe)
 Binary8p4se(4.0 ≐ 0x50)
 Binary8p4se(-3.25 ≐ 0xcd)
 Binary8p4se(2.25 ≐ 0x49)

julia> Exp(Binary8p4se, RNE_SatNone, A)
4-element Vector{Binary8p4se}:
 Binary8p4se(0.40625 ≐ 0x35)
 Binary8p4se(56.0 ≐ 0x6e)
 Binary8p4se(0.0390625 ≐ 0x1a)
 Binary8p4se(9.0 ≐ 0x59)
```

For pure (non-stochastic) specs, unary and binary array calls run as **table gathers**: the first call builds and caches a 256-byte (unary) or 64 KiB (binary) result table for that exact (`op`, `formats`, `p`) specialization, and every later element costs a single lookup — measured at 0.27 ns/element unary, 0.5 ns/element binary. Ternary and stochastic calls run the scalar pipeline per element. Inspect or reset the cache with `table_bytes()` and `empty_tables!()`.

Sorting is special-cased: values compare through integer order keys, and vectors of Binary sort with an **$O(n)$ counting sort** installed as the default algorithm — about 8× the stock comparison sort at 64 K elements, `rev=true` included. `sort(A)` just works; NaN sorts last (first under `rev=true`), matching Base's conventions. `TotalOrder(x, y)` exposes the draft's total order directly (single NaN largest).

3.6 Blocks and scaled operations

A `Block{B,FS,FE}` is a scale in format FS and B elements in format FE; its represented values are `scale × element` (the MX-style scheme). Construct directly or quantize a tuple with the draft's algorithm, which picks the scale from the maximum finite `|element|` and projects each element against it:

```
julia> b = Block(Binary8pluf(4.0), Binary8p4se(1.5), Binary8p4se(-0.75),
               Binary8p4se(2.0), Binary8p4se(0.5))
Block{4, Binary8pluf, Binary8p4se}(Binary8pluf(4.0 ≐ 0x82), (...))

julia> ConvertFromBlock(Binary8p3se, RNE_SatNone, b) # decode scale×elem, project
(Binary8p3se(6.0 ≐ 0x4a), Binary8p3se(-3.0 ≐ 0xc6), Binary8p3se(8.0 ≐ 0x4c), Binary8p3se(2.0 ≐ 0x44))

julia> BlockReduceAdd(Binary8p4se, RNE_SatNone, b) # exact Σ scale·xi, then project
Binary8p4se(13.0 ≐ 0x5d)

julia> ConvertToBlockMaxAbsFinite(Binary8pluf, Binary8p4se, RNE_SatNone, RNE_SatNone,
                                (Binary8p4se(100.0), Binary8p4se(-12.0), Binary8p4se(0.5), Binary8p4se(3.0)))
Block{4, Binary8pluf, Binary8p4se}(Binary8pluf(64.0 ≐ 0x86), (...))
```

Every scalar operation lifts to blocks (`BlockAdd`, `BlockExp`, ..., taking blocks and a result scale) and to the scaled form `ScaledOp` (`ScaledAdd(fr, p, s1, x1, s2, x2)` — block size 1). Reductions: `BlockReduceAdd`, `BlockReduceMultiply`, and `BlockDotProduct(fr, p, bx, by)`, whose lane products and accumulation are **exact** before the single final projection — there is no hidden intermediate rounding. `ConvertToBlock(fs, fr, p, xs, s)` quantizes against a scale you supply. `BlockVector` stores many same-shape blocks in structure-of-arrays layout.

3.7 Packed storage

For memory-bound work, `PackedVector{F}` stores code points at `bitwidth(F)` bits per element (a `Binary5p2se` vector at 5 bits/element instead of 8):

```
julia> v = Binary5p2se.(rand(Xoshiro(3), 6) .* 4);

julia> pv = PackedVector(v); sizeof(pv.data)
8                # 6 × 5 bits = 30 bits → one 64-bit word

julia> pv[3] == v[3]
true
```

It is a full `AbstractVector{F}` (indexing, collect, iteration), and `vmap` accepts it directly, unpacking cache-friendly tiles internally. The rule is *store packed, compute unpacked*: there is deliberately no in-place packed arithmetic.

3.8 Conformance and κ -approximate implementations

`conformance()` returns the live declaration — formats, operations, mode vocabulary, the table specializations actually instantiated this session, and every registered approximation. `conformance_report()` prints it; `conformance_dict()` returns a plain nested `Dict` for JSON/TOML serialization.

The default API is bit-exact, always. If you *want* a faster inexact kernel, register it — and the registry measures your honesty:

```
julia> ftz = ftz_variant(:Exp, Binary8p4se, Binary8p4se, RNE_SatFinite); # Annex example

julia> impl = register_approx! (:my_fast_exp, :Exp, Binary8p4se, (Binary8p4se,),
                             RNE_SatFinite, ftz);

julia> kappa (:my_fast_exp)      # measured max code-point deviation, verified exhaustively
4.0
```

Declaring κ smaller than the measured deviation throws; NaN-mismatching implementations must be acknowledged with an explicit $\kappa = \text{NaN}$. Retrieve with `approx(:my_fast_exp)`, list with `list_approx()`, and measure anything yourself with `measure_kappa / codedistance`.

3.9 Performance guidance

- **Pass format types statically.** Through `const` bindings, type parameters, or function arguments, every entry point fully specializes (scalar `Add` $\approx 18\text{--}26$ ns, `project` ≈ 13 ns, zero allocations). A format type read from a **non-const global** forces Julia's dynamic dispatch on every call ($\sim 1\ \mu\text{s}$ for keyword calls); one function barrier `f(::Type{T}, ...)` where `{T}` restores full speed.
- **Bulk work belongs in array calls.** The table-gather kernels are $\sim 50\times$ the scalar path; the first call per specialization pays a one-time build (≈ 0.4 ms unary, tens of ms for 8×8 binary tables).
- **Stochastic and ternary array calls** run the scalar pipeline per element; pass an explicit `rng` for reproducibility.
- **Memory:** `PackedVector` for storage; `BlockVector` for many blocks.
- The reproducible benchmark suite lives at `benchmark/benchmarking.jl` and generates a full markdown report for your machine.

3.10 Environment switches

`ENV["ByteFloats_Float128"] = "disable"` (set before using `ByteFloats`) disables the internal `Float128` fast paths in favor of pure MPFR — results are bit-identical (tested); only build/oracle speed changes.

Chapter 4

User Examples

Worked, runnable examples. Every output shown was captured from a real session; seeds are fixed so you can reproduce them exactly.

4.1 Basic

One value, every rounding mode

The value 2.30078125 sits between the Binary8p4se grid points 2.25 and 2.5 (ulp = 0.25 in [2, 4]):

```
using ByteFloats

for μ in (NearestTiesToEven(), NearestTiesToAway(), TowardPositive(),
        TowardNegative(), TowardZero(), ToOdd())
    v = Convert(Binary8p4se, ProjSpec(μ, SatNone()), 2.30078125)
    println(rpad(string(typeof(μ)), 20), " → ", decode(v))
end
```

NearestTiesToEven	→ 2.25
NearestTiesToAway	→ 2.25
TowardPositive	→ 2.5
TowardNegative	→ 2.25
TowardZero	→ 2.25
ToOdd	→ 2.25

(ToOdd keeps 2.25 because its significand is already odd; try 2.05 to see it move.)

Enumerating a whole format

Small formats are small enough to look at in full — 16 code points for Binary4p2se, sorted by the total order (single NaN last):

```
decode.(sort(Binary4p2se.(0x00:0x0f)))    # broadcast the code-point constructor
```

```
[-Inf, -2.0, -1.5, -1.0, -0.75, -0.5, -0.25, 0.0,  
 0.25, 0.5, 0.75, 1.0, 1.5, 2.0, Inf, NaN]
```

This is a good way to build intuition for a format before committing to it: you can see the subnormal spacing, the binade structure, and the dynamic range.

Saturation in one line each

```
julia> w, two = Binary8p4se(200.0), Binary8p4se(2.0);    # MaxFinite is 224
```

```
julia> Multiply(Binary8p4se, RNE_SatNone, w, two)  
Binary8p4se(Inf ≡ 0x7f)
```

```
julia> Multiply(Binary8p4se, RNE_SatFinite, w, two)  
Binary8p4se(224.0 ≡ 0x7e)
```


4.2 General AI

Log-odds belief updating — and where tiny formats bite reasoning

Classic probabilistic reasoning: accumulate independent evidence as log-likelihood ratios. A sensor with hit rate 0.85 and false-alarm rate 0.30 contributes $\log(0.85/0.30) \approx 1.0415$ per alarm. In Binary8p3se that datum is 1.0 — and the running belief stalls exactly the way the gradient accumulator does in the Deep Learning section:

```
using ByteFloats

function logodds_demo(nalarms)
    llr = Binary8p3se(log(0.85 / 0.30))    # quantized evidence weight
    acc = Binary8p3se(0.0)
    for _ in 1:nalarms
        acc = Add(Binary8p3se, RNE_SatNone, acc, llr)
    end
    post(l) = 1 / (1 + exp(-l))
    exact = nalarms * log(0.85 / 0.30)
    (decode(llr), round(exact; digits=2), decode(acc),
     round(post(exact); digits=5), round(post(decode(acc)); digits=5))
end
logodds_demo(12)
```

```
(1.0, 12.5, 8.0, 1.0, 0.99966)
# llr datum, exact log-odds, quantized log-odds, exact posterior, quantized posterior
```

The accumulator is wrong by a third (8.0 vs 12.5 — near 8 the ulp is 2.0, so $8 + 1$ rounds straight back to 8), yet the *decision* is untouched: both posteriors are ≈ 1 . Threshold decisions need order, not magnitude — which is why coarse formats work in reasoning pipelines far past the point where their arithmetic looks broken. When magnitude does matter, the cures are the same as for gradients: a wider accumulator format, or stochastic rounding.

Fuzzy inference in an unsigned format

Membership degrees live in $[0, 1]$ and are never negative — a job for an unsigned finite format. Binary8p6uf gives a $[0, 15.5]$ range with 1/32 resolution at 1; the classic fuzzy connectives are registry operations (Minimum = Gödel t-norm, Maximum = s-norm, Multiply = product t-norm):

```
F = Binary8p6uf
trap(x, a, b, c, d) = clamp(min((x - a) / (b - a), 1.0, (d - x) / (d - c)), 0.0, 1.0)
warm = F(trap(26.0, 15, 20, 24, 28))      # membership of 26 °C in "warm"
hot  = F(trap(26.0, 24, 30, 100, 101))    # ... and in "hot"

(decode(warm), decode(hot),
 decode(Minimum(F, RNE_SatNone, warm, hot)),      # warm AND hot
 decode(Maximum(F, RNE_SatNone, warm, hot)),      # warm OR hot
 decode(Multiply(F, RNE_SatFinite, warm, hot)))   # product t-norm
```

```
(0.5, 0.3359375, 0.3359375, 0.5, 0.16796875)
```

Because every connective is a bit-exact registry op, a fuzzy rule base evaluated in Binary8p6uf is *reproducible across machines to the code point* — a property sampled float pipelines cannot promise.

Search heuristics: what quantized scores keep is the ordering

Game-tree and beam search consume evaluation scores only through comparisons. Quantizing 200 heuristic scores to 8 bits collapses them onto 78 distinct code points — yet the argmax survives, and sorting is exact (integer order keys, $O(n)$ counting sort, single NaN ordered last deterministically):

```
using Random
rng = Xoshiro(21)
scores = randn(rng, 200) .* 3
q = Binary8p4se.(scores)

(argmax(scores), argmax(decode.(q)), argmax(scores) == argmax(decode.(q)),
 decode.(sort(q; rev=true)[1:5]), length(unique(codepoint.(q))))
```

```
(140, 140, true, [8.0, 7.5, 6.5, 6.0, 6.0], 78)
```

The two 6.0s are the caveat: quantization creates *ties* the exact scores did not have. TotalOrder's deterministic tie behavior means the search expands the same node on every run — ties change which answer you get, not whether you can reproduce it.

4.3 Machine Learning

Quantizing a weight tensor and measuring the damage

```
using ByteFloats, Random, Statistics

rng = Xoshiro(42)
w = randn(rng, 10_000) .* 0.25      # typical trained-weight scale
q = Binary8p4se.(w)                 # project every weight
back = decode.(q)

mean((w .- back).^2), maximum(abs.(w .- back)), count(isinf, back) / length(back)
```

```
(4.41e-5, 0.0312, 0.0)              # MSE, max |error|, overflow fraction
```

The max error is exactly half an ulp of the largest binade the data reached — the worst case nearest rounding permits.

Picking a format: precision vs range

For the same Gaussian tensor, $K = 8$ formats trade significand bits against exponent range. RMSE doubles per lost significand bit; MaxFinite grows explosively:

```
rng = Xoshiro(7); x = randn(rng, 50_000)
for F in (Binary8p5se, Binary8p4se, Binary8p3se, Binary8p2se)
    back = [decode(Convert(F, RNE_SatFinite, xi)) for xi in x]
    println(rpad(formatname(F), 12), " rmse = ", round(sqrt(mean((x .- back).^2)); sigdigits=3),
            " maxfinite = ", decode(MaxFiniteOf(F)))
end
```

```
Binary8p5se  rmse = 0.0133  maxfinite = 15.0
Binary8p4se  rmse = 0.0266  maxfinite = 224.0
Binary8p3se  rmse = 0.0528  maxfinite = 49152.0
Binary8p2se  rmse = 0.103   maxfinite = 2.147483648e9
```

For unit-scale data, spend bits on precision; buy range only when your data needs it (or get it from a block scale — see below).

Stochastic rounding is unbiased where nearest is not

Nearest rounding maps 0.30078125 to 0.3125 every single time — a systematic +0.0117 bias. Stochastic rounding is right *on average*:

```
target = 0.30078125                                # v = 3/16 of an ulp above 0.296875
σ = ProjSpec(StochasticA{16}(), SatNone())
rng = Xoshiro(1)
m = mean(decode(Convert(Binary8p4se, σ, target; rng)) for _ in 1:200_000)
(decode(Binary8p4se(target)), m)
```

```
(0.3125, 0.300765)                                # RNE result vs stochastic mean ≈ target
```

This is the property that makes stochastic rounding matter for accumulating many small contributions (gradients, activations statistics) in a low-precision format.

4.4 Deep Learning

MX-style block quantization — and the staging pitfall

Block formats pair a shared scale with narrow elements, tracking dynamic range that a bare element format cannot. Here each row of W lives at a different scale, spanning $\sim 2^{16}$ across the matrix:

```
using ByteFloats, Random, Statistics

rng = Xoshiro(3)
W = randn(rng, 64, 64) .* 2 .* (2.0 .^ (collect(0:63) ./ 4)) # per-row scales

function mx_rows(W, ::Type{FST}, ::Type{FS}, ::Type{FE}, ::Val{B}) where {FST,FS,FE,B}
    n, m = size(W)
    [begin
        # stage through a WIDE-RANGE format so nothing clamps before scaling
        seg = ntuple(k -> Convert(FST, RNE_SatFinite, W[i, (j-1)*B + k]), Val(B))
        ConvertToBlockMaxAbsFinite(FS, FE, RNE_SatNone, RNE_SatNone, seg)
    end for i in 1:n, j in 1:m ÷ B]
end

blocks = mx_rows(W, Binary8p2se, Binary8pluf, Binary8p4se, Val(32))
recon = [let b = blocks[i, (j-1) ÷ 32 + 1]
    decode(b.s) * decode(b.x[(j-1) % 32 + 1])
end for i in 1:64, j in 1:64]
plain = [decode(Convert(Binary8p4se, RNE_SatFinite, w)) for w in W]

relerr(A) = sqrt(mean(((W .- A) ./ max.(abs.(W), 1e-9)).^2))
relerr(plain), relerr(recon)
```

```
(0.616, 0.109) # plain 8p4se clamps the large rows; MX tracks them
```

Stage wide, then block-quantize

ConvertToBlockMaxAbsFinite takes already-Binary elements. If you stage your Float64 data through the *element* format first, SatFinite clamps the large values **before** the block scale can absorb them, and blocks buy you nothing — we measured exactly that (0.617 vs 0.616) with Binary8p4se staging. Stage through a wide-range format (Binary8p2se above); the residual 0.109 here is the staging format's own precision, which bounds what any downstream scheme can keep.

At $B = 32$ with an 8-bit scale the storage cost is 8.25 bits/value.

Quantized dot products with one final rounding

BlockDotProduct computes every lane product and the accumulation *exactly*, then projects once:

```
rng = Xoshiro(9)
a64, b64 = randn(rng, 32), randn(rng, 32)
qb(v) = ConvertToBlockMaxAbsFinite(Binary8p1uf, Binary8p4se, RNE_SatNone, RNE_SatNone,
    ntuple(i -> Convert(Binary8p4se, RNE_SatFinite, v[i]), Val(32)))
dq = BlockDotProduct(Binary8p4se, RNE_SatNone, qb(a64), qb(b64))
(a64' b64, decode(dq))
```

```
(5.0634, 4.5)      # difference is input quantization only, never accumulation error
```

Why training loops like stochastic rounding: the swamping demo

Accumulate 400 gradient steps of 0.011 into a Binary8p3se accumulator. Under nearest rounding, the moment the accumulator dwarfs the increment, every add rounds back to the accumulator — it **stalls**. Stochastic rounding keeps absorbing the increments in expectation:

```
function accumulate_demo(nsteps, g)
    σ = ProjSpec(StochasticA{16}(), SatNone())
    rng = Xoshiro(11)
    acc_rne = Binary8p3se(0.0); acc_sto = Binary8p3se(0.0)
    for _ in 1:nsteps
        acc_rne = Add(Binary8p3se, RNE_SatNone, acc_rne, Convert(Binary8p3se, RNE_SatNone, g))
        acc_sto = Add(Binary8p3se, σ, acc_sto, Convert(Binary8p3se, σ, g; rng); rng)
    end
    (nsteps * g, decode(acc_rne), decode(acc_sto))
end
accumulate_demo(400, 0.011)
```

```
(4.4, 0.125, 5.0)  # exact sum, RNE accumulator (stalled!), stochastic accumulator
```

The stochastic result is noisy (5.0 vs 4.4 on this seed) but unbiased; the RNE result is *wrong by 35×* and no amount of steps will fix it. In practice you keep a wider accumulator when you can — and use stochastic rounding when you can't.

Activation functions are 256-byte lookup tables

For pure specs, a unary activation over an 8-bit format is a *finite function* — the whole nonlinearity is one 256-byte table, built bit-exactly through the scalar path and then gathered per element. This is precisely the activation LUT you would burn into an accelerator:

```
using ByteFloats, Random
empty_tables!()
rng = Xoshiro(4)
pre = Binary8p4se.(randn(rng, 4096) .* 2.5)      # pre-activations,  $\pm 2.5\sigma$ 
act = Tanh(Binary8p4se, RNE_SatNone, pre)         # table-gather kernel

(table_bytes(),
 round(count(v -> abs(decode(v)) == 1.0, act) / 4096; digits=3),
 length(unique(codepoint.(act))),
 decode(NextLessThan(one(Binary8p4se))))
```

```
(256, 0.384, 100, 0.9375)
# LUT size; fraction saturated to  $\pm 1$ ; distinct output codes; last value below 1
```

Two things worth staring at. First, **saturation is severe at this input scale**: 38% of pre-activations land exactly on ± 1 — under RNE, tanh reaches 1 as soon as the true value rounds past the midpoint of the last gap, and the entire saturated tail becomes indistinguishable to the next layer. (Under TowardNegative the asymptote is honored instead: tanh of any finite positive input projects to 0.9375, never 1 — the projection mode is part of the activation design space.) Second, the choice of nonlinearity changes *information retention* in code points, not just shape:

```
actS = Softplus(Binary8p4se, RNE_SatNone, pre)
(length(unique(codepoint.(actS))), table_bytes())
```

```
(61, 512)          # softplus keeps 61 distinct codes here; two LUTs now cached
```

Auditing an activation for a format is this cheap: enumerate all 256 inputs, look at the output histogram, count what survives.

Packing a quantized model

```
n = 1_000_000
model = [rawvalue(Binary5p2se, UInt8(rand(0:31))) for _ in 1:n]
pv = PackedVector(model)
(sizeof(model), sizeof(pv.data))
```

```
(1000000, 625000) # 8 bits → 5 bits per value; indexing and vmap work directly on pv
```

Chapter 5

Technical Guide

How ByteFloats.jl works inside: the encoding, the projection engine, the oracle and its correctness protocol, the performance layers, and the verification doctrine that holds it all together. Read the [User Guide](#) first; this page assumes its vocabulary.

5.1 Layer map

Source files load in dependency order, each layer speaking only downward:

layer	file	provides
formats	formats.jl	Binary{K,P,SGN,EXT}, the 120 named aliases, Group M queries
specs	projspec.jl	rounding/saturation singletons, ProjSpec{R,S}
codec	decode_encode.jl	decode (generated tables + bit-composed compute), encode, order keys, counting sort, Class, Next ops
engine	project.jl	round_to_precision (mask-based Float64 core + generic core), saturate, project, project_interval
ops	ops_scalar.jl	result-kind protocol, apply_op, the operation registry, both API registers
oracle	oracle.jl	ω -semantics for all 52 operations
tables	tables.jl	the pure-p result-table cache
kernels	kernels.jl	Shape-A gathers, Shape-B scalar loops, vmap
blocks	blocks.jl	Block, block/scaled ops, exact reductions
packed	packed.jl	sub-byte PackedVector storage
approx	approx.jl	κ measurement/registry, conformance declaration

5.2 Encoding and decoding

A value is its code point (UInt8). The bit layout is the draft's: sign (signed formats), biased exponent, trailing significand; one NaN at the negative-zero slot; no -0 ; $\pm\text{Inf}$ adjacent to the extremes in extended formats.

decode is a **@generated constant-tuple lookup**: per format, a 2^K -tuple of Float64 datums built once from the computational decode (so table and computation are correct by construction and asserted equivalent exhaustively). Constant inputs still fold — `maxfinite_datum(T)` is a compile-time constant — while runtime decode is a single indexed load (≈ 0.7 ns). The computational decode assembles the Float64 **by bits** (normalize with `leading_zeros`, place exponent and mantissa fields, reinterpret), valid because every datum's exponent sits deep inside Float64's normal range; Float64 is thereby the *exact* carrier for all datums, an invariant the suite checks against an independent big-float transliteration of the draft.

Ordering runs on **integer order keys**: a sign-magnitude fold into UInt16, monotone with the total order, NaN mapped to `typemax`. Same-format `TotalOrder`, `isless`, and the numeric comparisons are key comparisons (~ 1 ns); since a format has at most $2^K + 1$ distinct keys, vectors sort with an **$O(n)$ counting sort** installed via `Base.Sort.defaultg` (forward and reverse orderings; anything exotic falls back to the stock algorithm).

5.3 The projection engine

`project(fr, p, X; R, sticky)` is the single write path into a code point:

RoundToPrecision → Saturate → Encode

RoundToPrecision produces a `Rounded(kind, sign, S, Q)` — an exact scaled significand S and exponent Q per the draft's $Q = \max(\lfloor \log_2 |X| \rfloor, 1-B) - P + 1$. Two implementations, proven equivalent exhaustively:

- the **generic core** (`_rtp_core`) works on any carrier (`Float64`, `Float128`, `BigFloat`) via exact power-of-two scaling and a fraction v compared against the mode's decision points;
- the **mask-based Float64 core** (`_rtp_f64`) extracts sign/exponent/significand fields directly, represents v as a 128-bit fixed-point integer with an OR-mask sticky for bits shifted out, and evaluates every mode — including the stochastic $\lfloor v \cdot 2^N \rfloor$ comparisons and RNITE ties — as integer field tests. Specials, zeros, and (Convert-only-reachable) subnormal `Float64` inputs bail to the generic core.

Symbolic sticky is how enclosures talk to the engine: `sticky` $\in \{-1, 0, +1\}$ declares the true value to be the carried value plus an infinitesimal of that sign. The engine folds it into every comparison; the delicate case — true value just *below* a representable dyadic — decrements into the previous binade and sets $v = 1^-$ (encoded in the mask core as an all-ones fixed-point fraction, which reproduces every predicate including the RNITE tie behavior). This is what lets directed modes land exactly on asymptotes: $\tanh \rightarrow 1^-$ under `TowardNegative` projects to `NextLessThan(1)`, automatically.

Saturate classifies against the format's range with two integer comparisons per side: the extremal magnitude in canonical (S, Q) form is a constant-folded function of the type parameters, the rounded value and the extremum share one lexicographic (Q, S) order (subnormals and the lowest normal binade share the same Q), signed formats use sign-magnitude symmetry, unsigned underflow is just the sign bit, and an internal HUGEQ sentinel represents "finite but astronomically large" (the $v = 1^-$ image of an infinite endpoint) so directed `SatNone` clamps it to `MaxFinite` correctly. The draft's twenty saturation rows then map the classification to `as-is` / `MaxFinite` / `MinFinite` / `±Inf` / `NaN`.

Encode is pure integer bit assembly, including the significand-carry renormalization and the subnormal/normal field split.

5.4 The oracle and the result-kind protocol

Every operation's defined result is computed by `weval`, which returns one of five result kinds; `apply_op` fast-splits the common one and finishes the rest:

kind	meaning	finished by
<code>Float64</code>	exact (specials; representable arithmetic)	direct project
<code>Float128</code>	exact by width analysis	direct project (<code>Float128</code> carrier)
<code>BigExactF</code>	exact at 2200-bit precision (wide-spread tail)	project on <code>BigFloat</code>
<code>Enclose128F</code>	correctly-rounded <code>Float128</code> bracket	sticky agreement, MPFR fallback
<code>EncloseF</code>	MPFR directed enclosure $f(\text{prec})$, optional <code>Float128</code> pre-filter f_q , optional eager <code>Float64</code> estimate y_d	three-stage: $y_d \rightarrow f_q \rightarrow$ interval protocol

Two **rigor classes** govern every non-`Float64` path, and their arguments are never mixed:

Class R (unconditional). (a) Sums of decoded datums are *exactly representable* in Float128 whenever operand bits + exponent spread fit 113 bits — checked in advance by integer exponent arithmetic (Add at $\Delta E \leq 100$, FMA ≤ 92 , FAA span ≤ 98); beyond, the 2200-bit path takes over. (b) IEEE mandates correct rounding for division and sqrt at every binary width. Divide/Recip therefore use the **Float64** quotient directly: it is within half an ulp of truth, so it serves as EncloseF's eager yd estimate with no Float128 arithmetic on the path at all. Sqrt/RSqrt use the **Float128** CR result: an inexact nearest-CR value q brackets the truth in the *open* interval ($\text{prevfloat}(q)$, $\text{nextfloat}(q)$) — no accuracy assumption in either case. Exactness itself is detected by an fma residual test.

Class E (envelope-conditional). Faithful-but-not-CR evaluations stand in for an enclosure only inside a generous envelope, resolved by the two-sided sticky gate. Two stages, cheapest first. *Float64 stage*: for operations whose Float64 libm is faithful (≤ 1 ulp $\approx 2^{-52}$ relative — the exp/log families, the hyperbolics, forward trig inside the $|x| \leq 10^{15}$ reduction window, atan, asinh, the softplus composition), the eager estimate yd carries envelope $E = |yd| \cdot 2^{-45}$, $\geq 2^7$ slacker than any faithful-libm error (measured margin on this machine: $\geq 2^{6.6}$). *Float128 stage*: libquadmath's estimate $y = \text{fq}()$ carries $E = |y| \cdot 2^{-90}$, at least 2^{18} slacker than any published libquadmath bound. In both stages, if the sticky-projected endpoints agree, that code point is the answer; if not, the next stage (ultimately the MPFR ladder) decides — so an envelope failure can only cost speed, never correctness, unless both endpoints agree *and* the envelope is wrong, which the differential-build tests rule out empirically by building every standard table twice (Float128-first and pure-MPFR) and byte-diffing, and which the exhaustive oracle cross-check re-verifies per operation against the 3072-bit protocol.

The interval protocol (`project_interval`) is the termination backbone: evaluate the MPFR enclosure at precision p ; if $lo == hi$ the value is exactly representable — project it; otherwise project lo with `sticky = +1` and hi with `sticky = -1`; agreement (projection is monotone at fixed R) yields the answer; disagreement doubles p up to 4096 bits. Termination requires that the enclosure not be chasing a value the grid can actually hit — which is why the π -scaled operations carry **Niven peels**: for dyadic arguments, $\tan(\pi r)$ takes rational values only at the quarter-integers (± 1) and $\text{atan2}/\pi$ only on the diagonals ($\pm 1/4$, $\pm 3/4$); those cases are answered exactly before any enclosure is built, and Niven's theorem proves the peel set complete.

5.5 Tables and kernels

For pure specs, unary and binary operations are **finite functions** — 256 or 65 536 entries — so the kernel layer materializes them once per (op, formats, p) into a locked cache (`Dict{TableKey, Memory{UInt8}}`, double-checked locking, builds outside the lock) and serves every later array call as a gather: Shape-A, one load per element, measured 0.27 ns/elem unary and 0.5 ns/elem binary. Tables are built *through the scalar path*, so they inherit its bit-exactness; the suite asserts `table == scalar` over every entry. Ternary and stochastic calls take Shape-B — the fully specialized scalar pipeline per element, with the RNG resolved once per array.

5.6 Blocks: exactness without a superaccumulator

`blockdecode` produces each lane's `scale × element` exactly (≤ 17 -bit significands in Float64). Reductions then apply **span filters**: one integer pass over lane exponents decides whether the whole sum (or dot product, with ≤ 33 -bit lane products) is exactly representable in Float128; if so, a plain Float128 accumulation is the exact answer; if not, an exact big-float accumulation takes over. Either way there is exactly one projection, at the end. `ωBlockProject` follows the draft's special rows for the scale (NaN, 0, $\pm\text{Inf}$) and divides each element result by the scale through its own cheapest-first CR-bracket / enclosure cascade (exact Float64 quotient $\rightarrow$ CR Float128 $\rightarrow$ bracket/pre-filter $\rightarrow$ MPFR interval), mirroring the scalar quotient group's rigor arguments.

5.7 The κ registry and conformance

κ is the maximum code-point distance (along the total order) between an implementation's result and the defined result, over inputs with finite defined results; any mismatch on non-finite defined results makes $\kappa = \text{NaN}$. Because inputs are enumerable, `register_approx!` *measures* κ exhaustively at registration and rejects understatement — a declared bound is a verified property, not a promise. `conformance()` assembles the declaration live from the operation registry, the table cache, and the approximation registry.

5.8 Verification doctrine

The value sets are small enough that sampling is never necessary, so the suite enumerates: formats against an independent draft transliteration (14 679); ordering over all 2.5 M same-format pairs plus Next-op edge tables (7.6 M); every unary operation on every input against a 3072-bit protocol run; divide and the ternaries exhaustively; stochastic R-sweeps with directed-asymptote pins; table $\equiv$ scalar; blocks against a from-scratch reference composition; Float128 carrier $\equiv$ Float64 carrier and Float128-first $\equiv$ MPFR differential builds; the mask rounding core $\equiv$ the generic core over datums and reachable sums/products at boundary stochastic budgets $N \in \{45, 60\}$; packed round-trips; and κ /conformance behavior — ≈ 8.8 M assertions. Deterministic **specialization regressions** (concrete inferred return types at the public entry points, zero warm-path allocation) stand in for timing assertions.

5.9 Benchmark doctrine

Recorded after two measurement post-mortems: a benchmark closure over any non-const global measures Julia's dispatch machinery, not the code under test — and it distorts *ratios*, not just absolutes, because dispatch cost varies with call shape (a dynamic keyword call costs $\sim 1 \mu\text{s}$; a dynamic positional call far less; six unresolved interior sites cost six times one). The rules the shipped benchmark/benchmarking.jl enforces structurally: format types enter as type parameters; operands come from untimed setup; functions retrieved reflectively pass through argument barriers to specialize; and a preflight aborts the run if warm scalar paths allocate. The table-build section reports both the cold build (cache evicted per sample) and the steady-state warm cache hit, since callers amortize the former through the latter. Vary one binding per variant; verify specialization before believing a number.

5.10 Deliberate limitations

No implicit cross-format arithmetic (promotion is to Float64, explicitly). No in-place packed arithmetic. No hidden threading (kernels are single-threaded in this version). Irrational/Rational inputs to Convert are rejected rather than double-rounded silently. The Float128 machinery never changes results — disabling it (ByteFloats_Float128=disable) is a tested no-op semantically.

Chapter 6

Technical Examples

Internals-level recipes: introspecting the pipeline, verifying custom code against the oracle, measuring κ , and benchmarking without fooling yourself. Outputs are captured from real sessions. Several examples use unexported internals (`ByteFloats.round_to_precision`, `ByteFloats.project`, ...); those are stable enough to learn from but are not covered by semantic-versioning guarantees.

6.1 Basic

Watching RoundToPrecision work

`round_to_precision` returns the draft's exact (sign, S, Q) decomposition before saturation ever sees it:

```
using ByteFloats
using ByteFloats: round_to_precision

r = round_to_precision(4, 8, NearestTiesToEven(), 2.30078125, 0, 0) # P=4, bias=8
(r.sign, r.S, r.Q, r.S * 2.0^r.Q)
```

```
(1, 9, -2, 2.25)          #  $S \cdot 2^Q = 9/4$ : the nearest P=4 value
```

Symbolic sticky: projecting "just below" a number

The engine accepts `sticky` $\in \{-1, 0, +1\}$ meaning "the true value is the carrier plus an infinitesimal of this sign". This is how enclosure endpoints and asymptotes ($\tanh \rightarrow 1^-$) are projected exactly:

```
using ByteFloats: project
project(Binary8p4se, ProjSpec(TowardNegative(), SatNone()), 1.0; sticky=-1)
```

```
Binary8p4se(0.9375  $\equiv$  0x3f)    # == NextLessThan(1.0): the engine crossed the binade
```

Tables are the scalar path, memoized

```
using ByteFloats: get_table
empty_tables!()
tbl = get_table(:Exp, Binary8p4se, Binary8p4se, RNE_SatNone)
(tbl[Int(0x45) + 1],
 codepoint(Exp(Binary8p4se, RNE_SatNone, rawvalue(Binary8p4se, 0x45))),
 table_bytes())
```

```
(0x52, 0x52, 256)          # table entry ≡ scalar result; one 256-byte table cached
```

Order keys

Ordering is integer arithmetic: a sign-magnitude fold, monotone with the total order, NaN at the top. This is what comparisons and the $O(n)$ counting sort run on:

```
using ByteFloats: order_key
[(v, order_key(v)) for v in (Binary8p4se(-1.0), Binary8p4se(0.0),
                             Binary8p4se(1.0), Binary8p4se(NaN))]
```

```
-1.0 → 64      0.0 → 129      1.0 → 193      NaN → 65535
```

6.2 General AI

Ranking safety: an exhaustive monotonicity audit of score conversion

Decision and ranking systems consume scores through *order*. If scores are produced in one format and compared in another, the conversion must be monotone — and formats are small enough to prove it by enumeration rather than trust it. Walk every Binary8p3se datum in total order and check that its Binary8p4se image never goes backward on the integer order keys:

```
using ByteFloats
using ByteFloats: order_key

function monotone_conversion(::Type{From}, ::Type{To}) where {From, To}
    prev = nothing
    for v in sort(From.(0x00:UInt8(2^bitwidth(From) - 1)))
        isnan(decode(v)) && continue
        g = Convert(To, RNE_SatNone, v)
        prev !== nothing && order_key(g) < order_key(prev) && return false
        prev = g
    end
    true
end
monotone_conversion(Binary8p3se, Binary8p4se)
```

```
true
```

Run this for every (From, To, p) triple your system actually uses; it is a few hundred integer comparisons per triple. A ranking pipeline whose conversions all pass this audit cannot invert a preference by changing formats — a guarantee no amount of spot-testing provides.

κ -safe decision margins for approximate evaluators

Search under a compute budget often wants a cheap, approximate evaluation function. The κ registry turns "how approximate?" into a *measured* code-point bound — and code-point bounds compose into a decision rule: **if two defined evaluations differ by more than 2κ code points along the total order, the approximate evaluator cannot invert their comparison.** Verify the rule exhaustively for a $\kappa = 2$ evaluator:

```
using ByteFloats: order_key

fast(x) = (r = Exp(Binary8p4se, RNE_SatNone, x);
           isfinite(decode(r)) ? NextGreaterThan(NextGreaterThan(r)) : r)
κ, exhaustive = measure_kappa(fast, :Exp, Binary8p4se, (Binary8p4se,), RNE_SatNone)

function margin_audit(fast, κ)
    codes = [rawvalue(Binary8p4se, UInt8(c)) for c in 0:255]
    safe = violations = 0
    for a in codes, b in codes
        da, db = Exp(Binary8p4se, RNE_SatNone, a), Exp(Binary8p4se, RNE_SatNone, b)
        (isnan(decode(da)) || isnan(decode(db))) && continue
        codedistance(da, db) > 2κ || continue
        safe += 1
        (order_key(fast(a)) < order_key(fast(b))) ==
            (order_key(da) < order_key(db)) || (violations += 1)
    end
    (safe, violations)
end
(κ, exhaustive, margin_audit(fast, κ)...)

```

```
(2.0, true, 49522, 0)
```

49,522 operand pairs clear the 2κ margin, and the approximate evaluator agrees with the defined ordering on every one of them — zero violations, exhaustively. Inside the margin, comparisons are genuinely undecidable at this κ ; a search can treat sub-margin comparisons as ties to expand, or escalate those few nodes to the exact evaluator. Either way the pruning is *provably* sound, with κ measured at registration rather than promised.

6.3 Machine Learning

Verifying a quantizer exhaustively against an independent reference

Formats are small enough that "test on a few points" is never necessary. Here every in-range datum of Binary8p3se is projected into Binary8p4se and checked against a brute-force nearest search under 256-bit arithmetic (distance agreement; the draft's tie rule is the projection engine's job and is pinned elsewhere in the suite):

```
using ByteFloats

function ref_nearest_distance(::Type{T}, x) where {T}
    fins = [v for v in (rawvalue(T, UInt8(c)) for c in 0:255) if isfinite(decode(v))]
    minimum(abs(setprecision(() -> BigFloat(decode(v)) - BigFloat(x), BigFloat, 256))
            for v in fins)
end

function verify_quantizer()
    ok = true
    for c in 0x00:0xff
        x = decode(rawvalue(Binary8p3se, c))
        (isfinite(x) && abs(x) <= decode(MaxFiniteOf(Binary8p4se))) || continue
        got = Convert(Binary8p4se, RNE_SatNone, x)
        ok &= abs(decode(got) - x) == Float64(ref_nearest_distance(Binary8p4se, x))
    end
    ok
end
verify_quantizer()
```

```
true
```

This pattern — enumerate the inputs, compare against an independently written reference — is how the entire package is tested, and it is available to *your* quantization code at trivial cost.

The κ workflow, including the part where it says no

Register an approximate kernel and the registry measures it; understate the bound and it refuses:

```
step2(x) = (r = Exp(Binary8p4se, RNE_SatNone, x);
            isfinite(decode(r)) ? NextGreaterThan(NextGreaterThan(r)) : r)

measure_kappa(step2, :Exp, Binary8p4se, (Binary8p4se,), RNE_SatNone)
```

```
(2.0, true)           #  $\kappa$  = 2 code points, verified over all 256 inputs
```

```
register_approx!(:cheater, :Exp, Binary8p4se, (Binary8p4se,), RNE_SatNone, step2;  $\kappa$ =1)
```

```
ERROR: ArgumentError: declared  $\kappa$  = 1.0 understates measured  $\kappa$  = 2.0 – registration rejected
```

Exporting the conformance declaration

```
d = conformance_dict()           # plain nested Dict{String,Any}
(d["package"], length(d["formats"]), length(d["operations"]),
 [s["op"] for s in d["cached_specializations"]])
```

Serialize `d` with any JSON/TOML writer to attach a machine-readable conformance statement to experiment artifacts.

6.4 Deep Learning

Proving your fused kernel exact: BlockDotProduct vs 512-bit truth

The block layer promises *one* projection with exact lane arithmetic. Trust, then verify — against big-float truth, over random blocks with mixed scales and an honest special-value mix:

```
using ByteFloats, Random

rng = Xoshiro(5)
mk() = Block(Binary8p4se, 2.0^rand(rng, -3:3),
             ntuple(_ -> rawvalue(Binary8p4se, UInt8(rand(rng, 0:255))), 32))

function verify_dots(trials)
    for _ in 1:trials
        bx, by = mk(), mk()
        lx = [decode(bx.s) * decode(v) for v in bx.x]
        ly = [decode(by.s) * decode(v) for v in by.x]
        (any(!isfinite, lx) || any(!isfinite, ly)) && continue
        truth = setprecision() -> sum(BigFloat(lx[i]) * BigFloat(ly[i]) for i in 1:32),
                                     BigFloat, 512)
        got = BlockDotProduct(Binary8p4se, RNE_SatNone, bx, by)
        ref = setprecision() -> ByteFloats.project(Binary8p4se, RNE_SatNone, truth), BigFloat, 512)
        codepoint(got) == codepoint(ref) || return false
    end
    true
end
verify_dots(200)
```

```
true
```

The same shape verifies any custom fused kernel you write: compose the truth in BigFloat, project once, compare code points.

Stochastic rounding, audited: the full-R sweep

Every stochastic projection is a deterministic function of the draw R , so its *distribution* is checkable exactly. For $x = 2 + 3/64$ in Binary8p4se the fraction is $v = 3/16$ of an ulp, so StochasticA{4} must round up for exactly 3 of the 16 draws:

```
σ4 = ProjSpec(StochasticA{4}(), SatNone())
x = 2.0 + 3/64
count(decode(ByteFloats.project(Binary8p4se, σ4, x; R)) == 2.25 for R in 0:15)
```

```
3
```

Sweeping R like this turns "is my stochastic pipeline unbiased?" from a statistical question into an exhaustive one — the pattern the shipped test suite uses.

An accelerator-style activation, κ -measured: hard-tanh vs Tanh

Hardware activation units often ship piecewise approximations. The κ machinery makes the substitution honest: measure the approximation's worst code-point deviation from the defined activation, exhaustively, and register it under that measured bound. Hard-tanh — `clamp(x, -1, 1)` — as a stand-in for Tanh:

```
one4 = Binary8p4se(1.0)
hardtanh(x) = Clamp(Binary8p4se, RNE_SatNone, x, Negate(one4), one4)

κ, exhaustive = measure_kappa(hardtanh, :Tanh, Binary8p4se, (Binary8p4se,), RNE_SatNone)
(κ, exhaustive)
```

```
(4.0, true)           # worst deviation: 4 code points, verified on all 256 inputs
```

Where is it worst? At `x = 1.0`: hard-tanh returns 1.0 while the defined `tanh(1.0)` is 0.75 — four code points along the total order. The registration round-trip, including the conformance surface:

```
impl = register_approx! (:hardtanh_act, :Tanh, Binary8p4se, (Binary8p4se,),
                        RNE_SatNone, hardtanh; κ=4)
(κappa(:hardtanh_act), κappa_measured(impl), :hardtanh_act in list_approx())
```

```
(4.0, 4.0, true)      # declared = measured; conformance_report() now lists it
```

(`unregister_approx! (:hardtanh_act)` removes it.) Declaring $\kappa=3$ would be *rejected* — understatement is impossible by construction. The decision of whether a $\kappa = 4$ activation is acceptable now belongs to your accuracy budget, not to hope: combined with the margin rule from the General AI section, any two pre-activations whose defined Tanh images sit more than 8 code points apart keep their order under hard-tanh, provably.

Benchmarking without measuring the dispatcher

The package's benchmark doctrine, in one snippet (needs the benchmark/ environment for Chairmarks). Format types enter as **type parameters**; operands come from Chairmarks' *untimed* setup; and if you retrieve functions reflectively (getfield(ByteFloats, op)), pass them through an argument barrier so they specialize:

```
using Chairmarks, ByteFloats, Random
using Statistics: median

function bench_add(::Type{T}) where {T<:Binary}           # T: type parameter, not a global
    pool = [rawvalue(T, rand(UInt8)) for _ in 1:4096]
    @be (rand(pool), rand(pool)) (t -> Add(T, RNE_SatNone, t[1], t[2]))(_)
end

b = bench_add(Binary8p4se)
(round(median(b).time * 1e9; digits=1), median(b).allocs)
```

```
(16.3, 0.0)           # ns per full scalar Add, zero allocations
```

The 60× trap

The same call with T read from a non-const global measures $\sim 1 \mu\text{s}$ — Julia's dynamic keyword dispatch, not this package. Two project post-mortems trace to exactly this mistake; the shipped benchmark/benchmarking.jl asserts specialization (zero warm-path allocation) before it believes any number, and so should yours.

Chapter 7

Adding Operations

How to add a new operation to ByteFloats.jl so that it inherits everything the existing catalog has: bit-exact defined results, the full projection-mode vocabulary, generated scalar/array/block surfaces, table caching, exhaustive test coverage, benchmark pickup, and a truthful conformance declaration. Read the [Technical Guide](#) first — this page assumes its vocabulary (the result-kind protocol, the two rigor classes, the interval protocol, the yd envelope stage).

The architecture is registry-driven: for the supported arities, **most of the surface is generated**. Adding an operation is mostly (1) one registry line and (2) one rigorous `weval` method — plus the *duties* that keep the package's guarantees true. Every subsection below states the duties explicitly; they are not optional.

7.1 Universal duties (every new operation)

1. **Special rows are spec, not optimization.** Write out the NaN/±Inf/zero rows explicitly in `weval`, in draft style, before any general evaluation. Mark any behavior the draft text under-determines with `[interp]`.
2. **Result-kind protocol.** `weval` must return one of `Float64` | `Float128` | `BigExactF` | `EncloseF` | `Enclose128F`, with `Float64`/ `Float128` reserved for *provably exact* results. Keep the return union narrow (ideally `{Float64, EncloseF}`) — a wide union defeats inference and costs allocations (measured: a 4-type union turned a 50 ns op into 240 ns with 2 allocations).
3. **Termination (the Niven duty).** `project_interval` terminates only if the enclosure is not chasing a value the projection grid can actually hit. Any input whose *exact* result is a representable dyadic must be **peeled or detected** before an enclosure is built. For transcendental ops this is usually a finite set of special rows (prove completeness, as the π -family does with Niven's theorem); for algebraic ops it is an exactness test.
4. **Envelope justification.** If you supply an eager yd estimate, you owe an error bound: the true value must lie within $|yd| \cdot 2^{-45}$ (`_F64_RELEXP`). Acceptable bases, strongest first: IEEE-CR hardware ops ($+$, $\sqrt{}$: $\leq \frac{1}{2}$ ulp, unconditional); short compositions of CR ops ($\leq \sim 1.5$ ulp); faithful libm (≤ 1 ulp, an empirical-but-published claim); short faithful compositions with condition number ≤ 1 ($\leq \sim 3$ ulp). **Measure it:** sample your estimator against ≥ 256 -bit MPFR truth over the op's domain, including the treacherous regions (near zeros of the result, domain boundaries), and demand $\geq 2^6$ slack under 2^{-45} . The same duty applies to a `Float128` fq estimator against its 2^{-90} envelope. Beware **argument-error amplification**: an estimate computed as $f(g(x))$ where g rounds (e.g. $\sin(\pi \cdot r)$ in `Float64`) can be catastrophically worse near zeros of f than a native result-domain evaluation ($\sin(\pi r)$) — measure the composition you actually ship.
5. **Ladder validity.** The MPFR closure must return a genuine directed enclosure. Whole-expression set rounding is valid only when the composition is monotone in every intermediate (state the argument in a comment, as `Softplus` does); otherwise round each step in the correct direction explicitly (as `_mpfr_rsqr` does for the anti-monotone $1/\sqrt{x}$).

6. **Gates.** Run the full suite. The unary exhaustive xhp gate and the coverage check pick up registry ops automatically; extend the hand-curated lists where applicable (monotonicity ops, binary exhaustive sweeps). Then regenerate the benchmark report — `bench_scalar_ops` iterates the registry lists, so your op appears in all four operand-class tables automatically; **add a `_SAFE_DOMAINS` entry** in `benchmark/benchmarking.jl` if the op is argument-restricted, or the `safe-args` table will use the oracle-derived fallback pool.

7.2 Scalar operations

The registry lives in `src/ops_scalar.jl`. Adding a name to `_UNARY_OPS` / `_BINARY_OPS` / `_TERNARY_OPS` causes, with no further work:

- registration in `OP_REGISTRY` (group `:B/:C/:A` per the loop defaults — override with an explicit `register_op!` if the default group is wrong);
- the generated spec-register method `Op(fr, p, operands...; rng, R)` and the same-format convenience `Op(x...)`;
- generated array methods routing through `vmap` (Shape-A tables for pure `p` at arity ≤ 2 , Shape-B scalar loops otherwise);
- generated `BlockOp` and `ScaledOp` variants (`src/blocks.jl`);
- the export (`src/ByteFloats.jl` loops the registry);
- table cacheability, conformance listing, and test-harness coverage.

What is *not* generated is the mathematics: `weval` in `src/oracle.jl`.

Adding a unary operation

Worked example: **Cbirt** (cube root), chosen because it exercises the hardest duty — exactness detection for an algebraic op.

Step 1 — registry (`src/ops_scalar.jl`): append `:Cbirt` to `_UNARY_OPS`. The loop registers it as group `:B` with arity 1.

Step 2 — ω -semantics (src/oracle.jl), with every duty discharged:

```
# Exact-cube detection (termination duty):  $x = \pm m \cdot 2^{(3k)}$  with  $m$  an odd perfect
# cube  $\Rightarrow$   $\text{cbrt}(x)$  is a Float64-exact dyadic that project_interval must never
# chase. Oracle-path cost is irrelevant; correctness of the peel is everything.
function _exact_cbrt(x::Float64)
    fr, e = frexp(abs(x))          #  $|x| = fr \cdot 2^e$ ,  $fr \in [0.5, 1)$ 
    m = Int64(ldexp(fr, 53)); e -= 53 #  $|x| = m \cdot 2^e$  exactly
    tz = trailing_zeros(m); m >= tz; e += tz
    e % 3 == 0 || return nothing
    r = round(Int64, cbrt(Float64(m)))
    for c in max(r - 1, 1):(r + 1) # cbrt is faithful  $\Rightarrow$  true root  $\in r \pm 1$ 
        Int128(c)^3 == Int128(m) && return flpsign(ldexp(Float64(c), e + 3), x)
    end
    nothing
end

function weval(::Val{:Cbrt}, x::Float64)
    isnan(x) && return NaN          # special rows first: they are spec
    isinf(x) && return x           #  $\text{cbrt}(\pm\infty) = \pm\infty$ 
    iszero(x) && return 0.0
    c = _exact_cbrt(x)
    c === nothing || return c      # exact dyadic result — peeled
    # yd: Float64 cbrt is faithful ( $\leq 1$  ulp  $\approx 2^{-52}$ ; measure it)  $\Rightarrow \geq 2^7$  slack
    # under the  $2^{-45}$  envelope. fq: libquadmath cbrtq under the  $2^{-90}$  envelope.
    # Ladder: cbrt is monotone  $\Rightarrow$  whole-expression directed rounding encloses.
    _encl1(cbrt, x; fq=() -> cbrt(Float128(x)), yd=cbrt(x))
end
```

Step 3 — optional surface: a Base veneer (Base.cbrt(x::Binary) = Cbrt(x) next to the others in ops_scalar.jl), a docstring, and — since cbrt is monotone — its symbol in the test suite's monotonicity list.

Step 4 — validate: envelope measurement for cbrt (include tiny and huge x; cbrt has no dangerous regions — the result is never near zero except at zero, which is peeled), then the full suite: the exhaustive xhp gate now checks Cbrt on every code point of the reference format under four modes against the 3072-bit protocol, automatically.

Adding a binary operation

Worked example: **LogAddExp** — $\log(e^x + e^y)$ — chosen because it exercises composed estimators and the ladder-monotonicity argument.

Step 1 — registry: append :LogAddExp to _BINARY_OPS (default group :C; the special five arithmetic names get :A).

Step 2 — ω -semantics. One generic evaluator serves all three precisions — Float64 (yd), Float128 (fq), and BigFloat (the ladder):

```
# Stable form: h + log1p(exp(l - h)), h = max, l = min. Every step is monotone
# nondecreasing in (x, y), so whole-expression directed rounding in _mpfr2 is a
# valid enclosure (the same argument Softplus records).
_logaddexp(a, b) = (h = max(a, b); l = min(a, b); h + log1p(exp(l - h)))

function weval(::Val{:LogAddExp}, x::Float64, y::Float64)
    (isnan(x) | isnan(y)) && return NaN
    (x == Inf || y == Inf) && return Inf          #  $\infty$  dominates
    x == -Inf && return y                        #  $\log(0 + e^y) = y$ , exactly
    y == -Inf && return x
    # yd: two faithful libm calls plus additions, condition  $\leq 1$  throughout
    # (result  $\geq \max(x, y) \Rightarrow \leq \sim 3$  ulp  $\approx 2^{-50} - 2^5$  slack; measure it, including
    #  $x \approx y$  (where  $\log1p(\exp(0))$  dominates) and widely separated operands.
    _encl2(_logaddexp, x, y;
        fq=() -> _logaddexp(Float128(x), Float128(y)),
        yd=_logaddexp(x, y))
end
```

Termination note: can $\logaddexp(x, y)$ be exactly dyadic for dyadic operands with the special rows peeled? $\log(e^x + e^y) = d$ requires $e^x + e^y = e^d$ — by Lindemann-Weierstrass this forces relations that dyadic arguments cannot satisfy except on the peeled rows, so no further peel is needed; **record that argument in a comment**. When you cannot prove such a statement for your op, hunt for exact cases the way `_exact_cbrt` does.

Step 3 — tests: binary ops are not in the unary exhaustive gate; add a divide-style exhaustive sweep (256×256 operand pairs against a ladder-only reference, a handful of modes) to the suite, and an envelope-sanity loop entry if you introduced a new libm dependence.

Adding a ternary operation

Worked example: **FMS** — fused multiply-subtract, $x \cdot y - z$ — chosen because the cheapest correct implementation is *delegation*, and delegation is a first-class technique here.

Step 1 — registry: append `:FMS` to `_TERNARY_OPS` (group `:A` by the loop). Ternary ops never tabulate ($2^{\{3K\}}$ entries); the generated array method runs the Shape-B scalar loop, and stochastic p draws per element — all automatic.

Step 2 — ω -semantics by delegation to FMA's already-verified analysis (width thresholds, `_twosum` exactness, wide-spread fallback):

```
#  $x \cdot y - z \equiv \text{FMA}(x, y, -z)$ . Negation of  $z$  follows the Subtract convention:
# preserve NaN, and map  $-0$  to  $+0$  so the single-zero datum model is respected.
weval(::Val{:FMS}, x::Float64, y::Float64, z::Float64) =
    weval(Val{:FMA}, x, y, isnan(z) ? z : (iszero(z) ? 0.0 : -z))
```

Delegation inherits FMA's result kinds, its exactness thresholds, *and* its test pedigree — but verify the sign algebra of your reduction on the special rows (here: $\text{FMS}(\infty, 1, \infty)$ must be NaN, which the delegation produces because $\text{FMA}(\infty, 1, -\infty)$ is $\infty - \infty$). Add the delegated op to the ternary exhaustive sweep in the suite.

Adding a quaternary operation

Arity 4 is **not yet plumbed** — the generation loops, kernels, and block layer handle arities 1–3. Adding a quaternary op is therefore two tasks: extend the plumbing (once), then add the op. Worked example: **FMMA** — $x \cdot y + z \cdot w$, a two-term dot product.

Plumbing (one-time), four sites:

1. `src/ops_scalar.jl` — register manually after the arity loops (`register_op!{:FMMA, 4, :A}`), and add an `op.arity == 4` branch to the spec-register generation loop, mirroring the ternary branch with a fourth operand:

```
else # arity 4
  @eval begin
    @inline function $name(fr::Type{<:Binary}, p::ProjSpec,
                          x::Binary, y::Binary, z::Binary, w::Binary;
                          rng::MaybeRNG=nothing, R::Union{Nothing,Int}=nothing)
      apply_op($V(), fr, p, _drawR(p, rng, R),
              decode(x), decode(y), decode(z), decode(w))
    end
    @inline $name(x::T, y::T, z::T, w::T; kw...) where {T<:Binary} =
      $name(T, default_projspec(T), x, y, z, w; kw...)
  end
end
```

(`apply_op` and `weval` are already variadic — no change needed there.)

2. `src/kernels.jl` — a four-array Shape-B `vmap!` method (copy the ternary one, add operand D), and an arity-4 branch in the registry-generated array-surface loop.
3. `src/blocks.jl` — either an arity-4 branch in the `Block/Scaled` generation loop (four operand blocks, four `blockdecodes`) or an explicit skip (`op.arity > 3 && continue`) if a block form is not wanted; be deliberate.
4. `src/approx.jl` — `conformance_report` prints arities with `for a in 1:3`; widen to `1:4`.

The op itself follows the FAA/FMA width-analysis playbook exactly:

```
const _DE_FMMA = 92      # two exact ≤17-bit products: 18 + ΔE ≤ 113, margin 3

function weval(::Val{:FMMA}, x::Float64, y::Float64, z::Float64, w::Float64)
  (isnan(x) | isnan(y) | isnan(z) | isnan(w)) && return NaN
  (((iszero(x) && isinf(y)) || (isinf(x) && iszero(y))) ||
   ((iszero(z) && isinf(w)) || (isinf(z) && iszero(w)))) && return NaN
  p = x * y; q = z * w                                # exact: ≤8-bit significands
  if isinf(p) || isinf(q)
    (isinf(p) && isinf(q) && p != q) && return NaN
    return isinf(p) ? p : q
  end
  s, e = _twosum(p, q)
  e == 0.0 && return iszero(s) ? 0.0 : s                # exact in Float64
  (_f128() && !iszero(p) && !iszero(q) && _expdiff(p, q) <= _DE_FMMA) &&
    return Float128(p) + Float128(q)                  # exact by width
  BigExactF() -> setprecision() ->
    BigFloat(x) * BigFloat(y) + BigFloat(z) * BigFloat(w), BigFloat, _BIGP))
end
```

Testing duty is heavier at arity 4: the full cross-product is 2^{32} tuples, so the suite entry must be *sampled* (seeded, ladder-referenced, $\geq 10^5$ tuples across several modes) plus hand-picked exhaustion of the special-row algebra and the width-threshold boundary ($\Delta E \in \{91, 92, 93\}$ constructions). Document the sampling in the test, since it breaks the suite's "enumerate, never sample" norm — that exception must be visible, not silent.

7.3 Block operations

The block layer (src/blocks.jl) implements the draft's elementwise schema **once** — blockdecode $\rightarrow$ ω -op lanewise $\rightarrow$ blockproject — and generates every registry op's BlockOp/ScaledOp from it. The reductions are hand-written on a second shared pattern. Which subsection you need depends on whether your operation fits one of those two patterns.

Adding a scaled block operation

If the scalar operation is (or can be) a **registry op, the scaled form is free**: registering :Cbrt above already produced ScaledCbrt(fr, ρ , s, x) and BlockCbrt(fr, ρ , b, sr) with the draft's §5.4 semantics — decode scale $\times$ element exactly per lane, apply the ω -op, BlockProject against the result scale through the division cascade. **Prefer this route**; it is the package's non-divergence mechanism.

Write a scaled op by hand only when it is *not* an elementwise image of a registry op. The primitive to compose with is _bp_element(fr, ρ , R, res, Sdat) — it owns the draft's scale-special rows (NaN/0/ $\pm\text{Inf}$ scale) and the entire division-by-scale rigor cascade. Example, a scaled affine $s_1 \cdot x + s_2 \cdot y$ projected against a *unit* result scale:

```
function ScaledAffine(fr::Type{<:Binary},  $\rho$ ::ProjSpec,
                    s1::Binary, x::Binary, s2::Binary, y::Binary;
                    rng::MaybeRNG=nothing)
    a = weval(Val(:Multiply), decode(s1), decode(x))::Float64 # exact lanes
    b = weval(Val(:Multiply), decode(s2), decode(y))::Float64
    _bp_element(fr,  $\rho$ , _drawR( $\rho$ , rng, nothing), weval(Val(:Add), a, b), 1.0)
end
```

The ::Float64 assertions are load-bearing: lane decodes are exact by width (≤ 17 -bit significands), and the assertion turns that analysis into a runtime check. The Add result may be any kind in the protocol — _bp_element finishes all of them. Dividing by a scale of 1.0 is the identity fast path.

Adding an elementwise block operation

Same schema, whole-block granularity. The three obligations: decode operand blocks with blockdecode (exact), evaluate lanewise with weval (never with ad-hoc Float64 math), and finish with blockproject against the supplied result scale (never with per-lane project — the scale-division rows belong to blockproject). Example — BlockRelu, an elementwise $\max(x, 0)$ that reuses the verified MaximumNumber semantics:

```
"""BlockRelu(fr,  $\rho$ , b, sr): lanewise  $\max(x_i, 0)$  — NaN lanes propagate NaN per
MaximumNumber's number-preferring rows only when both operands are NaN, i.e.
never here; state your lane semantics this explicitly for any new op."""
function BlockRelu(fr::Type{<:Binary},  $\rho$ ::ProjSpec, b::Block{B}, sr::Binary;
                  rng::MaybeRNG=nothing) where {B}
    X = blockdecode(b)
    Z = ntuple(i -> weval(Val(:MaximumNumber), X[i], 0.0), Val(B))
    blockproject(fr,  $\rho$ , sr, Z; rng)
end
```

Add the name to the block-surface export list and to `conformance()`'s `blocknames` extension vector so the declaration stays truthful. Test against a from-scratch reference composition (the suite's existing block tests show the pattern) — element format $\times$ scale format $\times$ B, exhaustively where feasible.

Adding a reductive block operation

Reductions do **not** fit the elementwise schema; they follow the second pattern, visible in `BlockReduceAdd/BlockDotProduct`: (1) resolve the ∞/NaN **fold algebra** on `Float64` classifications first; (2) an integer **span filter** decides whether the whole reduction is exactly representable in `Float128` — if so, plain `Float128` accumulation is the exact answer; (3) otherwise a `BigExactF` exact accumulator at provably sufficient precision; (4) exactly **one** projection, at the very end, via `_finish`. Never round intermediates.

Worked example — **BlockSumOfSquares**, $\sum x_i^2$:

```

"""BlockSumOfSquares(fr, p, b): project( $\sum$  decode-lanei2) — one rounding total.
Fold algebra: any NaN lane  $\Rightarrow$  NaN; else any  $\pm\text{Inf}$  lane  $\Rightarrow$  +Inf (squares cannot
cancel — no opposite-infinity NaN case exists, unlike ReduceAdd); else exact."""
function BlockSumOfSquares(fr::Type{<:Binary}, p::ProjSpec, b::Block{B};
                           rng::MaybeRNG=nothing, R::Union{Nothing,Int}=nothing) where {B}
    X = blockdecode(b)
    res = if any(isnan, X)
        NaN
    elseif any(isinf, X)
        Inf
    elseif all(iszero, X)
        0.0
    else
        # span filter: lanes carry  $\leq 17$ -bit significands  $\Rightarrow$  squares  $\leq 34$  bits and
        # a square's exponent is  $2 \cdot (\text{lane exponent})$ ; the sum is exact in Float128
        # when  $34 + 2 \cdot \text{span} + \lceil \log_2 B \rceil + 1 \leq 113 \Rightarrow 2 \cdot \text{span} + \lceil \log_2 B \rceil \leq 78$ .
        if _f128() && 2 * _expspan(X) + _log2ceil(B) <= 78
            acc = Float128(0)
            for v in X
                q = Float128(v)
                acc += q * q                # exact squares, exact sum
            end
            acc
        else
            BigExactF() -> setprecision(BigFloat, _REDPREC) do
                acc = BigFloat(0)
                for v in X
                    acc += BigFloat(v)^2    # exact at _REDPREC
                end
                acc
            end
        end
    end
    _finish(fr, p, _drawR(p, rng, R), res)
end

```

The two lines that demand your care in any new reduction are the **fold algebra** (derive it from the draft's reduce definition on the extended reals — here the absence of an opposite-infinities NaN case is a *theorem about squares*, stated in the docstring) and the **span-filter inequality** (derive it from operand widths, write the derivation in the comment, and add boundary-condition tests at the threshold, exactly as the suite does for the existing `_DE_*` constants). Everything else — the exact accumulator, the single `_finish`, stochastic R plumbing — is pattern.

7.4 After any addition: the closing checklist

- Full test suite green, including your new sweep entries.
- Envelope measurements recorded for any new yd/fq estimator.
- Benchmark report regenerated; `_SAFE_DOMAINS` entry added if the op is argument-restricted; sanity-check the op's row in the safe-args table.
- Docstring on the public name; `[interp]` markers where you interpreted.
- `conformance_report()` shows the op (automatic for registry ops; manual block additions extend blocknames).

Chapter 8

API Reference

Docstrings for the documented surface of the package. The full export list is far larger (368 names, including the 120 format aliases and the generated operation and block registers); names without docstrings are covered descriptively in the [User Guide](#) and [Technical Guide](#), and completing per-name docstring coverage is tracked work.

8.1 Public API

ByteFloats.ByteFloats - Module.

ByteFloats

A conforming, performance-oriented Julia implementation of the IEEE P3109 draft standard for arithmetic formats for machine learning (bitwidths 3–8).

Bit-exact defined results on every default path; the projection engine (RoundToPrecision → Saturate → Encode) is the single write path into a code point; approximate fast paths exist only behind the explicit `κ` registry (`register_approx!` / `approx`), never substituted silently. See `conformance()` for the live declaration and `ByteFloats.draft_revision()` for the implemented draft.

Performance note

Pass format types through `const` bindings, type parameters, or function arguments. All exported entry points fully specialize when the format type is statically known (measured: a complete scalar `Add` ≈ 26 ns, `project` ≈ 13 ns, zero allocations). Calling through a **non-const global** format type forces dynamic dispatch on every call — measured at ~ 1 μ s per scalar keyword call — which is Julia semantics, not package overhead; a single function barrier (`f(::Type{T}, args...)` where `{T}`) restores full speed. The test suite pins the specialization properties (concrete return types, zero warm-path allocation) as deterministic regressions.

ByteFloats.RNE_SatFinite - Constant.

(`NearestTiesToEven`, `SatFinite`) — nearest-even with everything clamped to the finite range.

ByteFloats.RNE_SatNone - Constant.

(`NearestTiesToEven`, `SatNone`) — the package-wide default `p`.

ByteFloats.ApproxImpl - Type.

A registered `κ`-approximate implementation of one operation specialization.

ByteFloats.Binary – Type.

Binary{K,P,SGN,EXT} <: **AbstractFloat**

A P3109 floating-point value: a code point in the format with bitwidth $K \in 3:8$, precision P , signedness $SGN::\text{Bool}$ ($\text{true} = \text{Signed}$), domain $EXT::\text{Bool}$ ($\text{true} = \text{Extended}$, i.e. the datum set includes infinities).

The code point occupies the low K bits of the payload byte; the high $8-K$ bits are maintained zero as a representation invariant. Construct raw code points with $T(c::\text{UInt8})$ (validated code-point construction), $\text{rawvalue}(T, c)$ (unchecked kernel route), or $\text{Binary}\{\dots\}(\text{Val}(:\text{code}), x::\text{UInt8})$; construct from numeric values with $T(x::\text{Real})$ (default projection spec) or Convert . UInt8 is the one argument type meaning *code point*; all other Reals mean *value*.

ByteFloats.Block – Type.

Block{B, FS<:Binary, FE<:Binary}

A P3109 block ($s, [x_1 \dots x_B]$): scale factor in format FS , $B \geq 1$ elements in format FE (draft §5). *isbits*; blocks live in registers/stack.

ByteFloats.BlockVector – Type.

BlockVector{B,FS,FE} <: **AbstractVector**{**Block**{B,FS,FE}}

Structure-of-arrays storage: $\text{scales}::\text{Vector}\{FS\}$, $\text{elems}::\text{Matrix}\{FE\}$ ($B \times n$, column-major so each block's elements are one contiguous column).

ByteFloats.FPClass – Type.

Eight-way datum classification returned by `Class` (draft §4.13.1).

ByteFloats.NearestTiesToAway – Type.

Round to nearest; ties away from zero (draft §4.7.1).

ByteFloats.NearestTiesToEven – Type.

Round to nearest; ties to the even code point (IEEE default; draft §4.7.1).

ByteFloats.PackedVector – Type.

PackedVector{F<:Binary} <: **AbstractVector**{F}

Bit-packed storage of F code points at $\text{bitwidth}(F)$ bits per element. Construct with $\text{PackedVector}(A::\text{AbstractVector}\{F\})$; recover bytes with $\text{Vector}(pv)$ or $\text{collect}(pv)$. Memory: $\lceil n \cdot K / 64 \rceil$ words vs n bytes unpacked.

ByteFloats.ProjSpec - Type.

```
ProjSpec{R<:RoundingMode3109, S<:SaturationMode}
```

A projection specification $\rho = (\text{rounding mode}, \text{saturation mode})$, draft §4.2. Zero-size; construct as `ProjSpec(NearestTiesToEven(), SatNone())` or via the exported constants. Kernels specialize on its type.

ByteFloats.StochasticA - Type.

Stochastic rounding, variant A of draft §4.7.4: $\text{RoundAway} \iff \lfloor v \cdot 2^N \rfloor + R \geq 2^N$.

ByteFloats.StochasticB - Type.

Stochastic rounding, variant B: $\text{RoundAway} \iff \lfloor v \cdot 2^{(N+1)} \rfloor + (2R+1) \geq 2^{(N+1)}$.

ByteFloats.StochasticC - Type.

Stochastic rounding, variant C: $\text{RoundAway} \iff \text{RNITE}(v \cdot 2^N) + R \geq 2^N$.

ByteFloats.ToOdd - Type.

Round inexact results to the nearest *odd* code point (draft §4.7.3; sticky-friendly).

ByteFloats.TowardNegative - Type.

Directed rounding toward $-\infty$ (draft §4.7.2).

ByteFloats.TowardPositive - Type.

Directed rounding toward $+\infty$ (draft §4.7.2).

ByteFloats.TowardZero - Type.

Directed rounding toward zero — truncation (draft §4.7.2).

ByteFloats.BlockDotProduct - Method.

`BlockDotProduct` (draft §5.3.2). Lane products can carry 64 significant bits, so the products and their sum are formed in the exact accumulator, with the ∞/NaN fold algebra resolved on the `Float64` classifications first.

ByteFloats.BlockReduceAdd - Method.

`BlockReduceAdd` (draft §5.3.1): `project(reduce( $\omega\text{Add}$ , [0, X...]))`.

ByteFloats.BlockReduceMultiply - Method.

`BlockReduceMultiply` (draft §5.3.1): `project(reduce( $\omega\text{Multiply}$ , [1, X...]))`.

ByteFloats.Class - Method.

`Class(v) -> FPClass` (draft §4.13.1): classify v as `NaN`, $\pm\text{Inf}$, $\pm\text{normal}$, $\pm\text{subnormal}$, or `zero`.

ByteFloats.Convert - Method.

```
Convert(fr,  $\rho$ , x) -> fr
```

Draft §4.9 `Convert( $\langle fx, fr, \rho \rangle$ )`. Accepts `Binary`, IEEE floats (widened exactly to the `Float64` carrier), `Integer` (exact via a sufficiently wide `BigFloat`), and `BigFloat` (projected directly; the caller warrants the value is exact).

`ByteFloats.ConvertFromBlock` – Method.

`ConvertFromBlock` (§5.2.1): blockdecode, then project each element (no scale division).

`ByteFloats.ConvertToBlock` – Method.

`ConvertToBlock` (§5.2.2): decode elements, `BlockProject` against the supplied scale.

`ByteFloats.ConvertToBlockMaxAbsFinite` – Method.

`ConvertToBlockMaxAbsFinite` (§5.2.3): $S = \text{reduce}(\omega_{\text{MaximumFinite}}, [\text{NaN}, |X_1| \dots])$, project S under ps , then `BlockProject` the elements under p .

`ByteFloats.NextGreaterThan` – Method.

`NextGreaterThan(v)` (draft §4.16): the least datum greater than v in the total order — one step up the code lattice, with $\text{NaN} \rightarrow \text{NaN}$ and $\text{MaxFinite}/+\text{Inf} \rightarrow \text{NaN}$ at the top. `Base.nextfloat` on `Binary` is this operation.

`ByteFloats.NextLessThan` – Method.

`NextLessThan(v)` (draft §4.16): the greatest datum less than v in the total order — one step down the code lattice, with $\text{NaN} \rightarrow \text{NaN}$ and $\text{MinFinite}/-\text{Inf} \rightarrow \text{NaN}$ at the bottom. `Base.prevfloat` on `Binary` is this operation.

`ByteFloats.RoundOf` – Function.

Draft-named accessor: `RoundOf(p)` — the rounding mode of p (§4.2).

`ByteFloats.SatOf` – Function.

Draft-named accessor: `SatOf(p)` — the saturation mode of p (§4.2).

`ByteFloats.TotalOrder` – Method.

`TotalOrder⟨fx,fy⟩` (draft §4.12.1): $x \leq y$ in the total order (single NaN largest). Same-format comparisons run on the integer order key (bitops plan K1) — proven equivalent to the decode order exhaustively over all pairs; cross-format keys are not comparable, so mixed formats keep the decode path.

`ByteFloats.approx` – Method.

Retrieve a registered approximate implementation (callable via `.fn`).

`ByteFloats.bitwidth` – Method.

Format bitwidth K (3–8).

`ByteFloats.codedistance` – Method.

Code-point distance along the total order between two values of one format.

`ByteFloats.conformance` – Method.

```
conformance() -> ConformanceDeclaration
```

The package's draft-§4.6-style conformance declaration, derived live from the operation registry, the table cache (the specializations actually instantiated), and the approximate-implementation registry. Serialize with `conformance_dict` or render with `conformance_report`.

`ByteFloats.conformance_dict` - Function.

Nested `Dict{String,Any}` form of the declaration — serializable by any JSON/TOML writer.

`ByteFloats.conformance_report` - Function.

Human-readable conformance report.

`ByteFloats.decode` - Method.

```
decode(v::Binary) -> Float64
```

ω Decode (draft §4.7.2). Float64 is the universal exact carrier for $K \leq 8$ datums (≤ 8 -bit significands, $|\text{exponent}| \lesssim 2^7$); exactness is asserted by exhaustive test.

Implemented as a constant-tuple lookup (bitops plan K2): the per-format table is generated once from `_decode_compute` above, so the two are correct by construction and asserted equivalent exhaustively; constant inputs still fold.

`ByteFloats.empty_tables!` - Method.

Drop every cached table (they rebuild lazily on next use).

`ByteFloats.expbias` - Method.

Exponent bias: $2^{(K-P-1)}$ signed, $2^{(K-P)}$ unsigned.

`ByteFloats.expsbitwidth` - Method.

Width of the exponent field in bits: $(K - \text{signbit}) - (P - 1)$.

`ByteFloats.formatname` - Method.

`formatname(T)` — the draft §3.2 name of a format type.

`ByteFloats.ftz_variant` - Method.

```
ftz_variant(op, fr, fl, p) -> fn
```

Build the Annex-style approximate unary implementation: compute the defined result, then flush subnormal results to zero or `MinNormalOf(fr)`, whichever is nearer, ties toward zero (magnitude-symmetric for signed formats). Returned `fn` is suitable for `register_approx!`; for `fr` with precision P its κ is $2^{(P-2)}$ when $P \geq 2$ (largest flushed-to-zero subnormal) and 0 when $P = 1$ (no subnormals exist).

`ByteFloats.isextended` - Method.

Whether the format's domain is Extended (datum set includes infinities).

`ByteFloats.issigned` - Method.

Whether the format is Signed (has a sign bit and negative datums).

`ByteFloats.list_approx` - Method.

Sorted names of all registered κ -approximate implementations.

ByteFloats.measure_kappa - Method.

```
measure_kappa(fn, op, fr, argformats, p; max_exhaustive=2^22, rng, samples=2^20)
  -> (κ::Float64, exhaustive::Bool)
```

Measure κ of $\text{fn}(x::\text{argformats}[1], \dots)::\text{fr}$ against the defined results of draft operation op under p . Enumerates the full input cross-product when it has at most max_exhaustive points (always true for $\text{arity} \leq 2$); otherwise measures on samples uniform draws and reports $\text{exhaustive} = \text{false}$.

ByteFloats.nrandbits - Method.

Number of random bits N consumed per projected element; 0 for pure modes.

ByteFloats.projmode - Method.

Translate a Base.RoundingMode to its RoundingMode3109 counterpart (identity on the latter). Used only at API boundaries; internals always carry RoundingMode3109.

ByteFloats.rawvalue - Method.

Unsafe raw constructor used by kernels after invariants are established.

ByteFloats.register_approx! - Method.

```
register_approx!(name, op, fr, argformats, p, fn; κ=nothing, kwargs...) -> ApproxImpl
```

Register fn as a named κ -approximate implementation of $\text{op}(\text{argformats} \rightarrow \text{fr}, p)$. κ is measured by enumeration (see `measure_kappa`); a declared κ smaller than the measured value is **rejected**. Omitting κ declares the measured value. NaN- κ implementations (non-finite mismatches) may be registered only by declaring $\kappa=\text{NaN}$.

ByteFloats.table_bytes - Method.

Total bytes currently held by the table cache.

ByteFloats.trailingsigbits - Method.

Trailing-significand width $P - 1$ (the stored fraction bits).

ByteFloats.unregister_approx! - Method.

Remove a registered κ -approximate implementation (no-op if absent).

ByteFloats.vmap! - Function.

```
vmap!(dest, Val(op), fr, p, A[, B[, C]]; rng) -> dest
vmap(op, fr, p, A...; rng)
```

Elementwise draft operation over arrays of Binary values, projecting into fr under p . Pure p with $\text{arity} \leq 2$ runs the Shape-A gather; ternary and stochastic specializations run the scalar path per element (each element drawing its own R).

ByteFloats.vmap – Method.

```
vmap(op, fr, p, pv::PackedVector; rng=nothing) -> Vector{fr}
```

Elementwise op over packed storage: unpack a 256-element tile into a byte scratch, run the ordinary byte kernel (vmap!), emit, repeat. Never computes on packed words directly — the deliberate compute-unpacked/store-packed boundary.

8.2 Documented internals

Unexported machinery documented in-source and referenced by the [Technical Guide](#) and [Technical Examples](#).

ByteFloats.BigExactF – Type.

Deferred exact result: `f()` returns the value as an exact `BigFloat`.

ByteFloats.Enclose128F – Type.

Correctly-rounded Float128 bracket (plan Site C, Class R): IEEE mandates correct rounding for Float128 /, sqrt, fma, so a nearest-CR result of a value known inexact brackets the truth in the *open* interval $(lo, hi) = (prevfloat(q), nextfloat(q))$ with no envelope assumption. `f` is the MPFR fallback for (near-impossible at $P \leq 8$) grid-straddling brackets.

ByteFloats.EncloseF – Type.

Deferred enclosure, resolved by `_finish` through up to three stages, cheapest first:

1. `yd` — an *eager* Float64 estimate (`NaN` $\Rightarrow$ absent) whose libm evaluation is faithful (≤ 1 ulp); the true value is taken to lie within $|yd| \cdot 2^{-45}$ (`_F64_RELEXP`), $\geq 2^7$ slacker than any faithful-libm error.
2. `fq` — a zero-argument Float128 estimator (plan Site D, Class E; `nothing` $\Rightarrow$ absent): true value within $|y| \cdot 2^{-90}$ of $y = fq()$ (`_F128_RELEXP`), a bound $\geq 2^{18}$ slacker than any published libquadmath transcendental error, discharged empirically by the differential-build tests.
3. `f(prec)` — the rigorous MPFR ladder: directed $(lo, hi)::NTuple{2, BigFloat}$ strictly bracketing the true value.

Each estimate stage returns only when the two-sided sticky projection of its envelope agrees on a single code point; any disagreement (or an absent/degenerate estimate) falls through to the next stage. Stage 3 always decides.

ByteFloats.Rounded – Type.

Result of `ωRoundToPrecision` in canonical integer form: a kind tag plus $sign \cdot S \cdot 2^Q$ for finite results (design §5.2). The saturation stage consumes this without ever re-touching the carrier value.

ByteFloats.TableKey – Type.

Value key for the table cache: `op` + format parameters + `p` parameters (all values, so the cache itself is type-stable and non-allocating to query).

Core.Type - Method.

```
(T::Type{<:Binary})(c::UInt8) -> T
```

Construct a value from its **code point** (validated: for $K < 8$, c must be $< 2^K$ or an `ArgumentError` is thrown; the range check folds away at $K = 8$ and for constant codes). `Binary5p3sf(0x08) == Binary5p3sf(1.0)`.

`UInt8` is the *only* argument type with code-point semantics — every other `Real` (including other `Integers`) constructs by projecting the numeric value, and `Convert` is numeric for all integers: `Binary8p4se(0x02)` is code point 2 (the second-smallest subnormal), while `Binary8p4se(2)` is the number 2.0. `rawvalue(T, c)` remains the unchecked kernel-internal route.

ByteFloats._bp_element - Method.

One element of ω BlockProject: the draft's S -special rows, then ω Divide + ω Project.

Fast-path cascade after the special rows, ordered cheapest-first by result kind: exact `Float64` quotient $\rightarrow$ CR `Float128` quotient (exact or one-ulp bracket) $\rightarrow$ CR-divided `Enclose128F` bracket $\rightarrow$ fq-composition envelope (2^{-89}) — each resolved by the two-sided sticky gate; any miss falls through to the rigorous MPFR interval at the bottom.

ByteFloats.apply_op - Method.

`apply_op(Val(op), fr, p, R, xs...)` — evaluate op 's ω -semantics on decoded `Float64` operands and project the result into `fr` under p (with random bits R).

ByteFloats.blockdecode - Method.

ω BlockDecode (draft §5.1.1): per lane ω Multiply(`decode(s)`, `decode(xi)`) — exact `Float64`.

ByteFloats.blockproject - Method.

ω BlockProject (draft §5.1.2) over a tuple of result kinds, scale supplied as a value.

ByteFloats.encode - Method.

```
encode(T, sign, S, Q) -> UInt8 (private; design §3.3)
```

ω Encode from canonical integer form: $\text{value} = \text{sign} \cdot S \cdot 2^Q$, with $S \in 0:2^P$ ($S == 2^P$ is the next-binade carry, draft §4.7.4 NOTE 4). Precondition: the value is in the datum set of T (guaranteed by `RoundToPrecision` ◦ `Saturate`).

ByteFloats.get_table - Function.

```
get_table(op, fr, f1, [f2,] p) -> Memory{UInt8}
```

Fetch (building and caching on first use) the complete result table for the pure- p specialization $op\langle fr; f1[, f2] \rangle$ under p : entry $c + 1$ (unary) or $(c1 \ll K2) + c2 + 1$ (binary) holds the result code point for those operand code points. Throws for stochastic p . @noinline by design: kernels call this once per array operation and index the returned table in their hot loop.

ByteFloats.project - Method.

```
project(T, ρ, X; R=0, sticky=0) -> T
```

Project a closed-extended-real carrier value into format T under ρ. $X::\text{Float64}$ must be *exact* (the Float64 fast path); $X::\text{BigFloat}$ likewise (or an interval endpoint with $\text{sticky} \in \{-1, +1\}$). R is the stochastic draw.

ByteFloats.project_interval - Method.

```
project_interval(T, ρ, f; R=0, maxprec=4096) -> T
```

Project the true value of f, where $f(\text{prec})$ returns an enclosure $(d::\text{BigFloat}, u::\text{BigFloat})$ with the true value in $[d, u]$ (endpoints from MPFR directed rounding). Rigor (design §9.2): if $d == u$ the value is exact — project it. Otherwise the true value lies in the *open* interval (d, u) (directed correct rounding of a non-representable value moves strictly); projection is monotone in the value for every mode at fixed R, so if $\text{project}(d, \text{sticky}=+1) == \text{project}(u, \text{sticky}=-1)$ that common code is the answer; otherwise a projection grid point sits inside the interval and precision escalates.

ByteFloats.unpack_tile! - Method.

Unpack $\text{pv}[\text{first}:\text{first}+\text{len}-1]$ into buf (byte scratch); tile-granular kernel entry.